

School of Engineering and Computer Science

ECE – 5730

Embedded System Design w/FPGAs

Fall 2022

Project Report on

"Image Display and

Touch Response using DE1SoC Board"

Submitted by:

Pradeep Potturi *Akram Mirza Baig* *Shaibal Saha* *Noor Ahmed*
Aqdas Mohammed *Shreyas Renukuntla* *Sunish Vadakkeveetil* *Michael Lee*

Submitted To:

Prof. Subramaniam Ganesan

Contents

1 Abstract 1
2 Introduction 2
3 Altera DE1SoC Board 3
4 Block Diagram of the Altera DE1SoC Board 6
5 Terasic TRDB_LTM LCD Display 10
5.1 Kit Contents 10
5.2 Connecting LTM to the Altera DE Board 11
5.3 Architecture of the LTM 11
5.4 Block Diagram of the LTM 13
5.5 Pin Description of the 40-pin Interface of LTM 14
5.6 The Serial Port Interface of the LCD Driver IC 14
5.7 Input timing of the LCD panel display function 17
5.8 The serial interface of the AD converter 19
5.9 LCD Timing Controller 21
6 Software Design 24
6.1 Application 1 – Chessboard Display 26
6.1.1 Design Approach: 26
6.1.2 Design Implementation: 26
6.2 Application 2 – BMP Image Display 27
6.2.1 Design Approach: 27
6.2.2 Design Implementation: 29
6.3 Application 3 – Coordinate Tracker Display 30
6.3.1 Design Approach: 30
6.3.2 Design Implementation: 30
7 Results 32
7.1 RGB Color Display 32
7.2 BMP Image Displays 35
7.3 Coordinate Tracker Display 39
8 Conclusion 42
8.1 Challenges 42

8.2 Future Work 43
9 References: 45
10 Code 46
10.1 Project.vhd 46
10.2 Touch_Irq_Detector 58
10.3 Three Wire Controller 61
10.4 Reset Delay 65
10.5 Lcd Timing Controller (RGB and Chessboard Pattern) 67
10.6 LCD Timing Controller (Displaying smiley image (100x400 resolution)) 85
10.7 Lcd SPI Controller 99
10.8 ADC SPI Controller 106
10.9 Seven Segment 114
10.10 Counter 116

List of Figures

Figure 1. DE1-SoC Board 3
Figure 2. Block Diagram of the Altera DE1SoC Board 6
Figure 3. The TRDB_LTM Package 10
Figure 4. Connecting the IDE Cable to the TRDB_LTM Board 11
Figure 5. Block Diagram of the LTM 13
Figure 6. The pin-out of the 40-pin connector 14
Figure 7 The serial interface of the LCD touch panel module and AD7843 15
Figure 8. The frame format and timing diagram of the serial port interface 16
Figure 9. LCD horizontal timing specification 19
Figure 10. Conversion timing of the serial port interface 21
Figure 11. Division of the LCD screen into 800x480 pixel resolution mode 22
Figure 12. Specifications for the horizontal timing 22
Figure 13. Specifications for the vertical timing 23
Figure 14. RTL View of Design 1 26
Figure 15. RGB Values from the Image 27
Figure 16. Color Pallet Image 28
Figure 17. Smiley Face Images 28

Figure 18. RTL View of Design 2 29
Figure 19. RTL View of Design 3 30
Figure 20. Vertical display of the RGB pattern 33
Figure 21. RGB color pattern display 33
Figure 22. Chess Board Pattern Display 34
Figure 23. LCD Display of Color Pallet 35
Figure 24. 1x4 display of the Smiley Face 36
Figure 25. Smiley face 1 Display in LCD 37
Figure 26. Smiley face 2 Display in LCD 37
Figure 27. Smiley face 3 Display in LCD 38
Figure 28. Smiley face 4 Display in LCD 38
Figure 29. Design 3 Picture 1 39
Figure 30. Design 3 Picture 2 40
Figure 31. Design 3 Picture 3 41
Figure 32. Future Architecture Design including SDRAM 44

List of Tables

Table 1: General Specifications of the LTM 12
Table 2: The timing parameters of the serial port interface 17
Table 3: LCD horizontal timing parameters 19
Table 4: Control word provided to the ADC via the DIN pin 20

1 Abstract

The final project for this class required each group of students to propose, design, plan, implement and deliver a project using the Altera DE1 SoC board, Quartus Prime II software, Terasic TRDB-LTM Touch Display, and any other necessary components.

For our project, we proposed to develop 3 different Applications

1. Chessboard Display
2. BMP Image Display
3. Coordinated Touch Tracker Display

In this project we will deal with the following components and modules

1. configuring the LCD display,
2. A/D conversion,
3. data request, data transfer
4. displaying color and
5. Touch count on the screen.

Keywords: Altera DE1 SoC board, Quartus Prime II software, Terasic TRDB-LTM Touch Display and A/D conversion

2 Introduction

For the design project for this class, we proposed three separate applications to implement. All three of the designs will make use of the DE1-SOC FPGA board along with the LTM display.

- The first project will be a simple project that will display a chessboard-like pattern on the display. When a touch is detected the screen will showcase other predefined images on the screen. The purpose of this is to practice with RGB values of the LCD and to also be able to detect touch.
- The second project will be display a bmp image onto the screen. The bmp image is split into a 10x10 array and the RGB values are stored in an array. Then the values are written to the LCD screen, and similar to the first design the screen will change when there is a touch detected.
- Lastly, the third design will take the coordinates of the touch that is detected and draw a red box in that area. Wherever the touch detection moves to, then that is where the red box will follow/move to.

All three of these designs were created with VHDL and allow for the practice of implementing the LCD screen along with the touch detection functions of the touch screen.

3 Altera DE1SoC Board

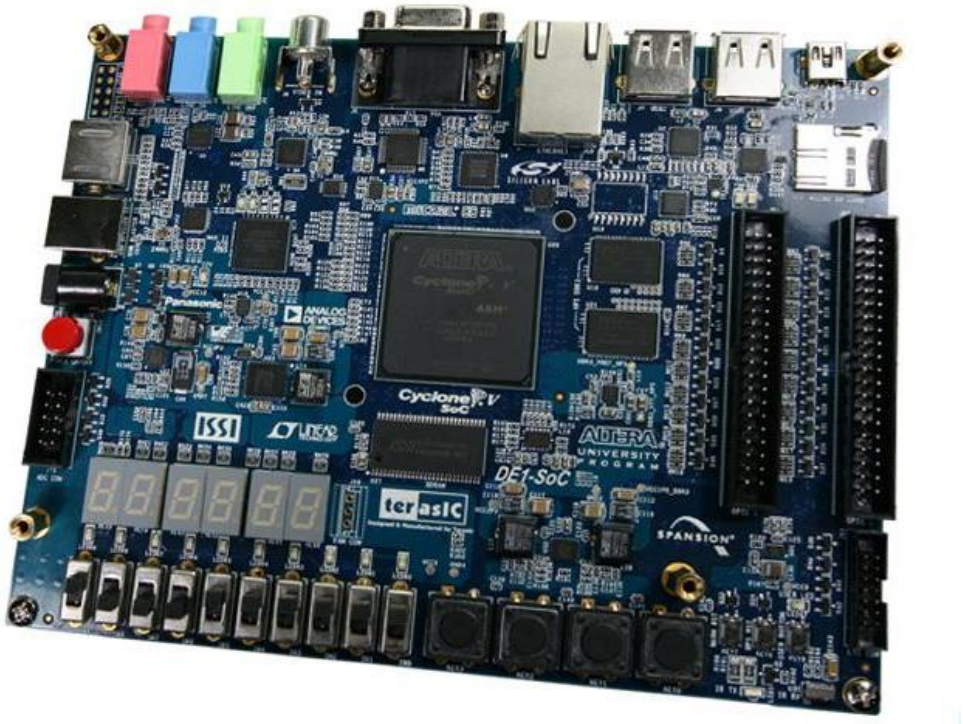


FIGURE 1. DE1-SOC BOARD

The DE1-SoC Development Kit presents a robust hardware design platform built around the Altera System-on-Chip (SoC) FPGA, which combines the latest dual-core Cortex-A9 embedded cores with industry-leading programmable logic for ultimate design flexibility. Users can now leverage the power of tremendous re-configurability paired with a high-performance, low-power processor system. Altera's SoC integrates an ARM-based hard processor system (HPS) consisting of processor, peripherals and memory interfaces tied seamlessly with the FPGA fabric using a high-bandwidth interconnect backbone. The DE1-SoC development board is equipped with high-speed DDR3 memory, video and audio capabilities, Ethernet

networking, and much more that promise many exciting applications. The DE1-SoC Development Kit contains all the tools needed to use the board in conjunction with a computer that runs the Microsoft Windows XP or later. The DE1-SoC board has many features that allow users to implement a wide range of designed circuits, from simple circuits to various multimedia projects. The following hardware is provided on the board:

- Altera Cyclone® V SE 5CSEMA5F31C6N device
- Altera serial configuration device – EPCS128
- USB-Blaster II onboard for programming; JTAG Mode
- 64MB SDRAM (16-bit data bus)
- pushbuttons
- 10 slide switches
- 10 red user LEDs
- Six 7-segment displays
- Four 50MHz clock sources from the clock generator
- 24-bit CD-quality audio CODEC with line-in, line-out, and microphone-in jacks
- VGA DAC (8-bit high-speed triple DACs) with VGA-out connector
- TV decoder (NTSC/PAL/SECAM) and TV-in connector
- PS/2 mouse/keyboard connector

- IR receiver and IR emitter
- Two 40-pin expansion headers with diode protection
- A/D converter, 4-pin SPI interface with FPGA ν HPS (Hard Processor System)
- 800MHz Dual-core ARM Cortex-A9 MPCore processor
- 1GB DDR3 SDRAM (32-bit data bus)
- 1 Gigabit Ethernet PHY with RJ45 connector
- 2-port USB Host, normal Type-A USB connector
- Micro SD card socket
- Accelerometer (I2C interface + interrupt)
- UART to USB, USB Mini-B connector
- Warm reset button and cold reset button
- One user button and one user LED
- LTC 2x7 expansion header

4 Block Diagram of the Altera DE1SoC Board

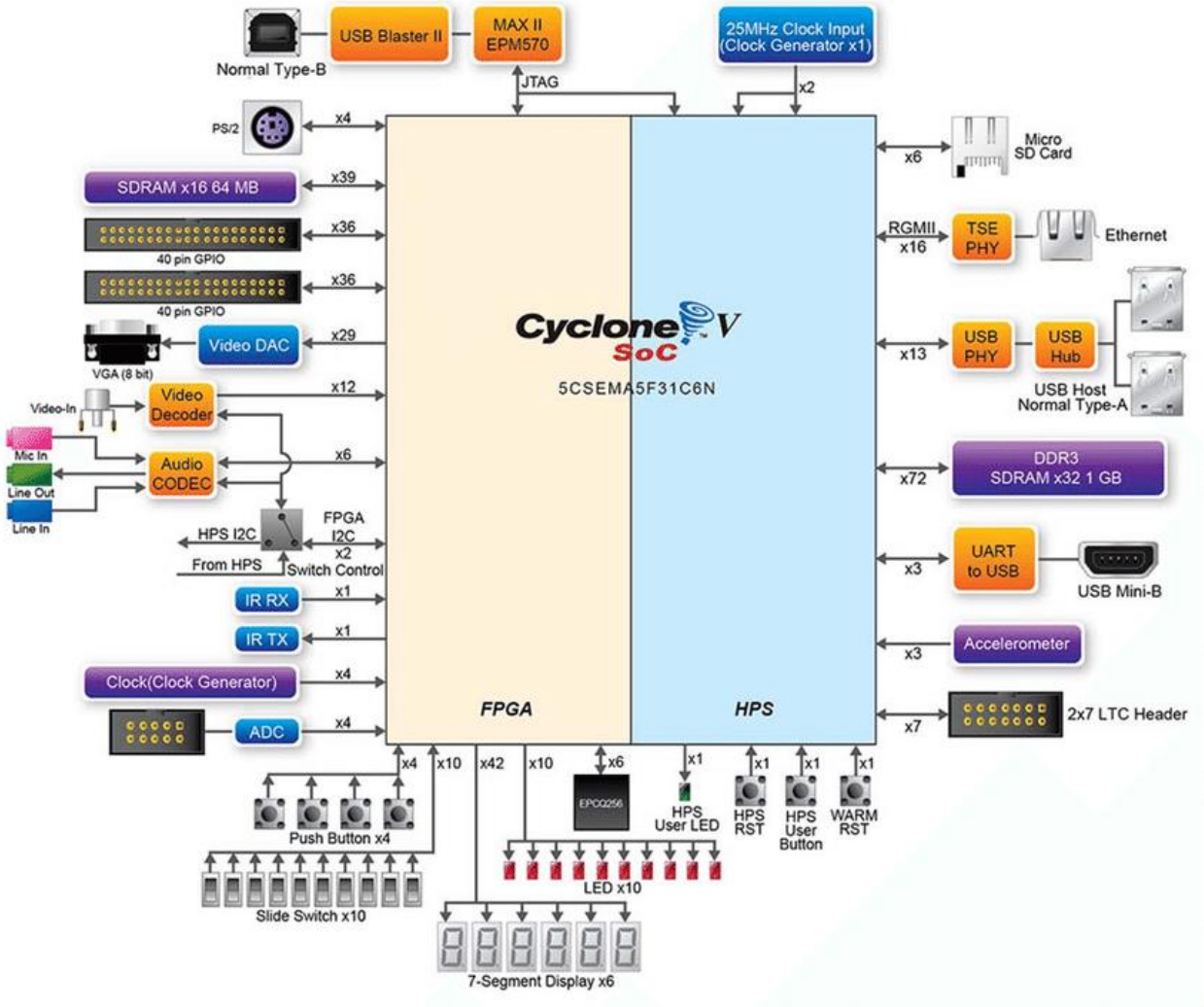


FIGURE 2. BLOCK DIAGRAM OF THE ALTERA DE1SOC BOARD

The details of each block are explained below:

FPGA Device:

• Cyclone V SoC 5CSEMA5F31 Device

• Dual-core ARM Cortex-A9 (HPS)

• 85K programmable logic elements

• 4,450 Kbits embedded memory

• 6 fractional PLLs

• 2 hard memory controllers

FPGA Configuration and Debug:

• Quad serial configuration device – EPCQ256 on FPGA

• Onboard USB-Blaster II (normal type B USB connector)

Memory Devices:

• 64MB (32Mx16) SDRAM on FPGA

• 1GB (2x256Mx16) DDR3 SDRAM on HPS

• Micro SD card socket on HPS 10

Communication:

• Two port USB 2.0 Host (ULPI interface with USB type A connector)

• UART to USB (USB Mini-B connector)

• 10/100/1000 Ethernet

• PS/2 mouse/keyboard

• IR emitter/receiver

• I2C multiplexer

Connectors:

• Two 40-pin expansion headers

• One 10-pin ADC input header

• One LTC connector (one Serial Peripheral Interface (SPI) Master, one I2C and one GPIO interface)

Display:

• 24-bit VGA DAC

Audio:

• 24-bit CODEC, Line-in, Line-out, and microphone-in jacks

Video Input:

• TV decoder (NTSC/PAL/SECAM) and TV-in connector

ADC:

• Fast throughput rate: 1 MSPS

• Channel number: 8

• Resolution: 12-bit

• Analog input range: 0 ~ 2.5 V or 0 ~ 5V as selected via the RANGE bit in the control register

Switches, Buttons, and Indicators:

• 5 user Keys (FPGA x4, HPS x1)

• 10 user switches (FPGA x10)

• 11 user LEDs (FPGA x10, HPS x 1)

• 2 HPS reset buttons (HPS_RESET_n and HPS_WARM_RST_n)

• Six 7-segment displays

Sensors:

• G-Sensor on HPS

5 Terasic TRDB_LTM LCD Display

The TRDB_LTM (LTM) Kit provides everything you need to develop applications using a digital touch panel on an Altera DE2/DE1 board. The kit contains complete reference designs and source code for implementing a photo viewer demonstration and a color pattern generator using the LTM kit and an Altera DE2/DE1 board. This chapter provides users key information about the kit.

5.1 Kit Contents

Fig 3. shows the picture of the TRDB_LTM package. The package includes:

1. The Terasic **L**CD **T**ouch Panel **M**odule **(LTM)** board
2. A 40-pin IDE cable
3. Complete reference design with source code



FIGURE 3. THE TRDB_LTM PACKAGE

5.2 Connecting LTM to the Altera DE Board

Please follow the two steps below to connect LTM to the Altera DE2/DE1 board:

1. Connect the IDE cable to the back of the LTM board, as shown in Fig 4.
2. Connect the other end of the IDE cable to your DE2/DE1 board's innermost expansion port (JP1) as shown.



FIGURE 4. CONNECTING THE IDE CABLE TO THE TRDB_LTM BOARD

5.3 Architecture of the LTM

This section will illustrate the architecture of the LTM including device features and block diagram.

Features

The feature set of the LTM is listed below:

1. Equipped with Toppoly TD043MTEA1 active-matrix color TFT LCD module.
2. Support 24-bit parallel RGB interface.

3. 3-wire register control for display and function selection.

4. Built-in contrast, brightness, and gamma modulation**.**

5. Converting the X/Y coordination of the touch point to its corresponding digital data via the Analog Devices AD7843 AD converter**.**

6. The general specifications of the LTM are listed below**:**

Item	Description	Unit
Display Size (Diagonal)	4.3	Inch
Aspect ratio	15:9	-
Display Type	Transmissive	-
Active Area (HxV)	93.6 x 56.16	mm
Number of Dots (HxV)	800 x RGB x480	dot
Dot Pitch (HxV)	0.039 x 0.117	mm
Color Arrangement	Stripe	-
Color Numbers	16Million	-

TABLE 1: **GENERAL SPECIFICATIONS OF THE LTM**

5.4 Block Diagram of the LTM

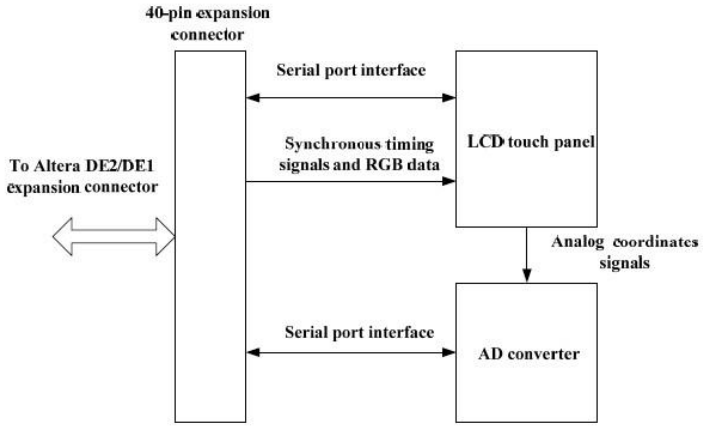


FIGURE 5. BLOCK DIAGRAM OF THE LTM

The LTM consists of three major components: LCD touch panel module, AD converter, and 40-pin expansion header. All of the interfaces on the LTM are connected to Altera DE2/DE1 board via the 40-pin expansion connector. The LCD and touch panel module will take the control signals provided directly from FPGA as input and display images on the LCD panel. Finally, the AD converter will convert the coordinates of the touch point to its corresponding digital data and output to the FPGA via the expansion header.

5.5 Pin Description of the 40-pin Interface of LTM

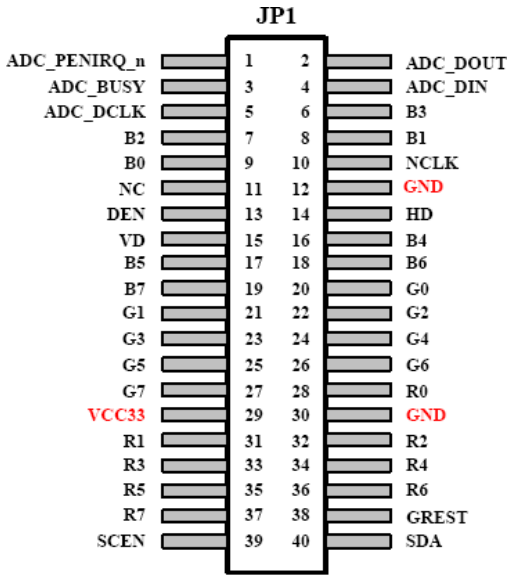


FIGURE 6. THE PIN-OUT OF THE 40-PIN CONNECTOR

5.6 The Serial Port Interface of the LCD Driver IC

The LCD and touch panel module on the LTM is equipped with a LCD driver IC to support three display resolution and with functions of source driver, serial port interface, timing controller, and power supply circuits. To control these functions, users can use FPGA to configure the registers in the LCD driver IC via serial port interface.

Also, there is an analog to digital converter (ADC) on the LTM to convert the analog X/Y coordinates of the touch point to digital data and output to FPGA through the serial port interface of the ADC.

Both LCD driver IC and ADC serial port interfaces are connected to the FPGA via the 40-pin expansion header and IDE cable.

Because of the limited number of I/O on the expansion header, the serial interfaces of the LCD driver IC and ADC need to share the same clock (ADC_DCLK) and chip enable (SCEN) signal I/O on the expansion header. To avoid both the serial port interfaces may interfere with each other when sharing the same clock and chip enable signals, the chip enable signal (CS), which is inputted into the ADC will come up with a logic inverter as shown in Figure

Users need to pay attention controlling the shared signals when designing the serial port interface controller. The detailed register maps of the LCD driver IC are listed in appendix chapter.

The specifications of the serial port interface of the LCD driver IC are described below.

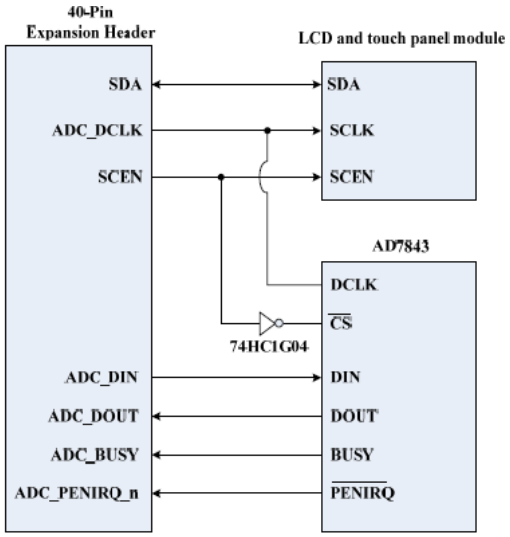


FIGURE 7 THE SERIAL INTERFACE OF THE LCD TOUCH PANEL MODULE AND AD7843

The LCD driver IC supports a clock synchronous serial interface as the interface to a FPGA to enable instruction setting. Please notice that in addition to the serial port interface signals, NCLK input should also be provided while setting the registers. Fig 8. and Table 2 show the frame format and timing diagram of the serial port interface. The LCD driver IC recognizes the start of data transfer on the falling edge of SCEN input and starts data transfer. When setting instruction, the TPG110 inputs the setting values via SDA on the rising edge of input SCL.

The first 6 bits (A5 ~ A0) specify the address of the register. The next bit means Read/Write command. “0” is write command. “1” is read command. Then, the next cycle is turn- round cycle.

Finally, the last 8 bits are for Data setting (D7 ~ D0). The address and data are transferred from the MSB to LSB sequentially. The data is written to the register of assigned address when “End of transfer” is detected after the 16th SCL rising cycles. Data is not accepted if there are less or more than 16 cycles for one transaction.

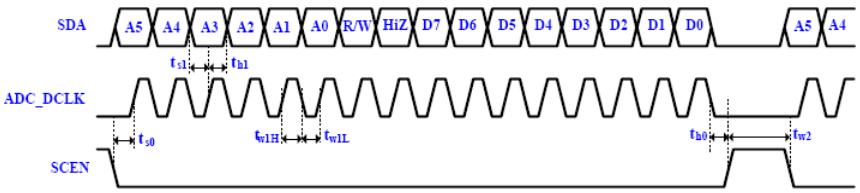


FIGURE 8. THE FRAME FORMAT AND TIMING DIAGRAM OF THE SERIAL PORT INTERFACE

Item	Symbol	Conditions	Min.	Max.	Unit
SDA Setup Time	ts0	SCEN to SCL	150		ns
	ts1	SDA to SCL	150		ns
SDA Hold Time	th0	SCEN to SCL	150		ns
	th1	SDA to SCL	150		ns
Pulse Width	tw1L	SCL pulse width	160		ns
	tw1H	SCL pulse width	160		ns
	tw2	SCEN pulse width	1.0		ns
Clock duty			40	60	%

TABLE 2: THE TIMING PARAMETERS OF THE SERIAL PORT INTERFACE

5.7 Input timing of the LCD panel display function

To determine the sequencing and the timing of the image signals displayed on the LCD panel, the corresponding synchronous signals from FPGA to the LCD panel should follow the timing specification.

Fig 9. illustrates the basic timing requirements for each row (horizontal) that is displayed on the LCD panel. An active-low pulse of specific duration (time ***hpw t*** in the figure) is applied to the horizontal synchronization (HD) input of the LCD panel, which signifies the end of one row of data and the start of the next. The data (RGB) inputs on the LCD panel are not valid for a time period called the hsync back porch (***hbp t***) after the hsync pulse occurs, which is followed by the display

area (***hd t***). During the data display area, the RGB data drives each pixel in turn across the row being displayed. Also, during the period of the data display area, the data enable signal (DEN) must be driven to logic high. Finally, there is a time period called the hsync front porch (***hfp t***) where the RGB signals are not valid again before the next hsync pulse can occur.

The timing of the vertical synchronization (VD) is the same as shown in the following figure, except that a vsync pulse signifies the end of one frame and the start of the next, and the data refers to the set of rows in the frame (horizontal timing). Table 3 show for different resolutions, the durations of time periods ***hpw t*** , ***hbp t*** , ***hd t*** , and ***hfp t*** for both horizontal and vertical timing. Finally, the timing specification of the synchronous signals is shown in the Table 3.4.

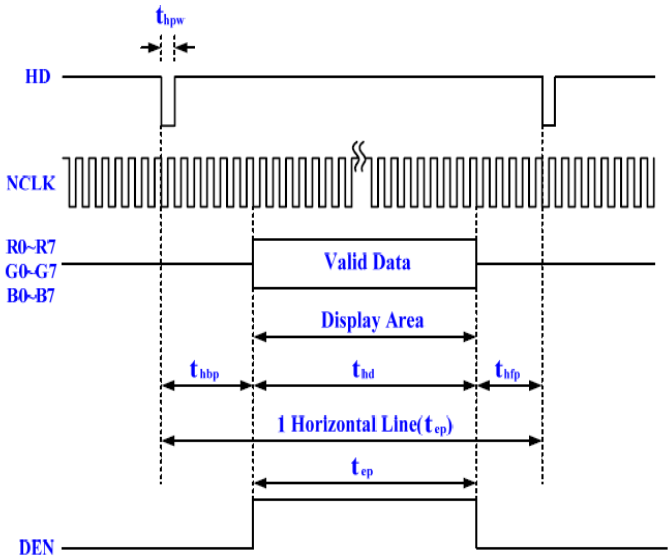


FIGURE 9. LCD HORIZONTAL TIMING SPECIFICATION

Parameter		Symbol	Panel Resolution			Unit
			800xRGBx480	480xRGBx272	400xRGBx240	
NCLK Frequency		FNCLK	33.2	9	8.3	MHz
Horizontal valid data		t_{hd}	800	480	400	NCLK
1 Horizontal Line		t_h	1056	525	528	NCLK
HSYNC Pulse Width	Min.	t_{hpw}	1			NCLK
	Typ.		-			
	Max.		-			
Hsync back porch		t_{hbp}	216	43	108	NCLK
Hsync front porch		t_{hfp}	40	2	20	NCLK
DEN Enable Time		t_{ep}	800	480	400	NCLK

TABLE 3: LCD HORIZONTAL TIMING PARAMETERS

5.8 The serial interface of the AD converter

The LTM also equipped with an Analog Devices AD7843 touch screen digitizer chip. The AD7843 is a 12-bit analog to digital converter (ADC) for digitizing x and y coordinates of touch points applied to the touch screen. The coordinates of the touch point stored in the AD7843 can be obtained by the serial port interface.

To obtain the coordinate from the ADC, the first thing users need to do is monitor the interrupt signal ADC_PENIRQ_n outputted from the ADC. By connecting a pull high resistor, the ADC_PENIRQ_n output remains high normally. When the touch screen connected to the ADC is touched via a pen or finger, the ADC_PENIRQ_n output goes low, initiating an interrupt to a FPGA that can then instruct a control word to be written to the ADC via the serial port interface.

The control word provided to the ADC via the DIN pin is shown in Table 4. This provides the conversion start, channel addressing, ADC conversion resolution, configuration, and power-down of the ADC. The detailed information on the order and description of these control bits can be found from the datasheet of the ADC in the *DATASHEET* folder on the LTM System CD-ROM.

MSB							LSB
S	A2	A1	A0	MODE	SER / $\overline{DEF}$	PD1	PD0

Bit	Mnemonic	Comment
7	S	Start Bit. The control word starts with the first high bit on DIN. A new control word can start every 15th DCLK cycle when in the 12-bit conversion mode, or every 11th DCLK cycle when in 8-bit conversion mode.
6-4	A2-A0	Channel Select Bits. These three address bits, along with the SER / $\overline{DEF}$ bit, control the setting of the multiplexer input, switches, and reference inputs.
3	MODE	12-Bit/8-Bit Conversion Select Bit. This bit controls the resolution of the following conversion. With 0 in this bit, the conversion has a 12-bit resolution, or with 1 in this bit, the conversion has a 8-bit resolution.
2	SER / $\overline{DEF}$	Single-Ended/Differential Reference Select Bit. Along with Bits A2– A0, this bit controls the setting of the multiplexer input, switches, and reference inputs.
1,0	PD1,PD0	Power Management Bits. These two bits decode the power-down mode of the AD7843.

TABLE 4: CONTROL WORD PROVIDED TO THE ADC VIA THE DIN PIN

Figure 10 shows the typical operation of the serial interface of the ADC. The serial clock provides the conversion clock and also controls the transfer of information to and from the ADC. One complete conversion can be achieved with 24 ADC_DCLK cycles. The detailed behavior of the serial port interface can be found in the datasheet of the ADC. Note that the clock (ADC_DCLK) and chip enable signals (SCEN) of the serial port interface

SHRAE the same signal I/O with LCD driver IC. Users should avoid controlling the LCD driver IC and ADC at the same time when designing the serial port interface controller. Also, because the chip enable signal (SCEN) inputted to the ADC comes up with a logic inverter, the logic level of the SCEN should be inverse when it is used to control the ADC.

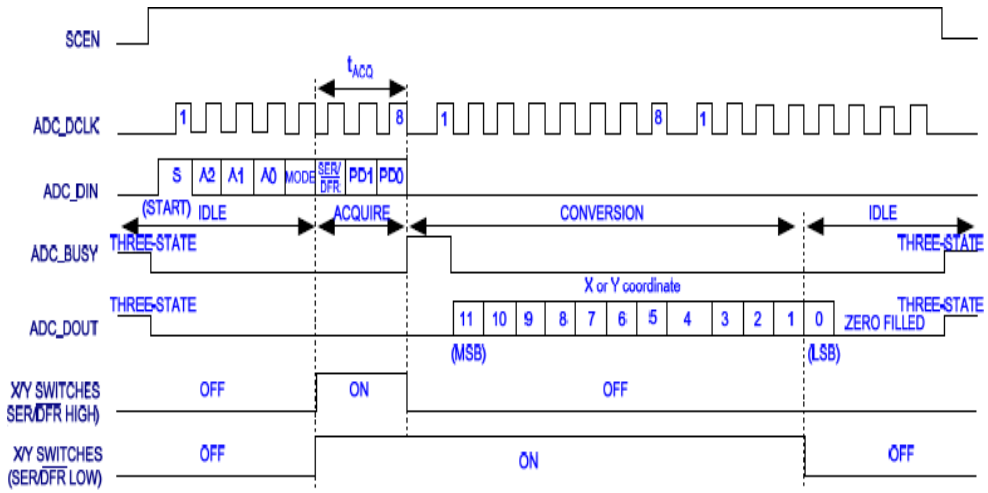


FIGURE 10. CONVERSION TIMING OF THE SERIAL PORT INTERFACE

5.9 LCD Timing Controller

First, let's look at how the LCD screen is divided in the 800x480 pixel resolution mode.

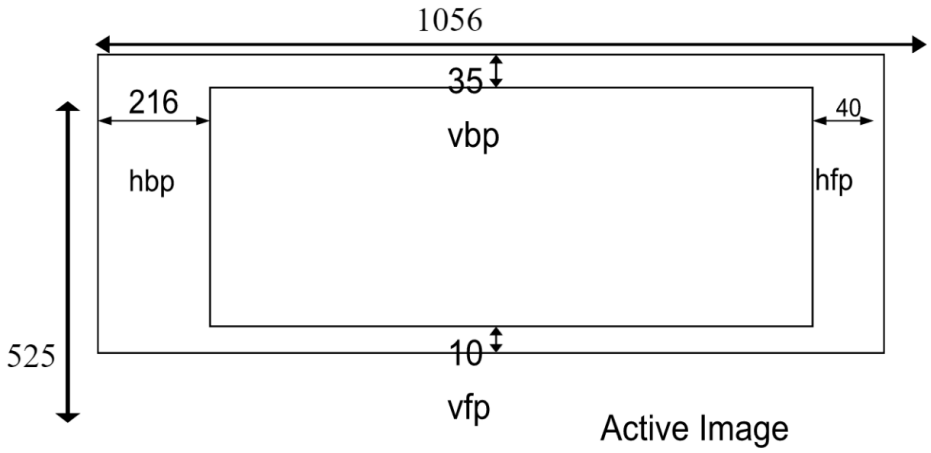


FIGURE 11. DIVISION OF THE LCD SCREEN INTO 800X480 PIXEL RESOLUTION MODE

This way the screen is divided is much like the screen of a VGA monitor. There is a horizontal and vertical back and front porches. In this region you should not be feeding the VGA's DAC any meaningful frame data. The active image region is where the active image should be displayed. Here are the timing specifications for the horizontal timing:

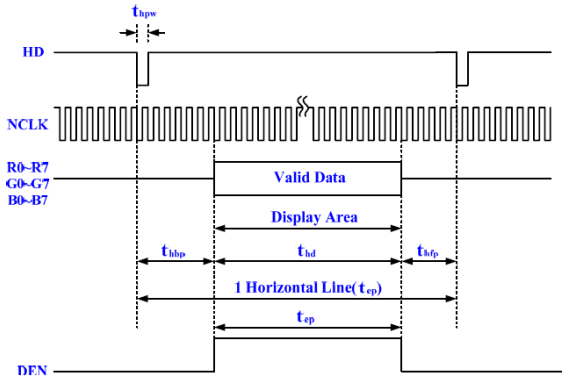


FIGURE 12. SPECIFICATIONS FOR THE HORIZONTAL TIMING

Here are the timing specifications for the vertical timing:

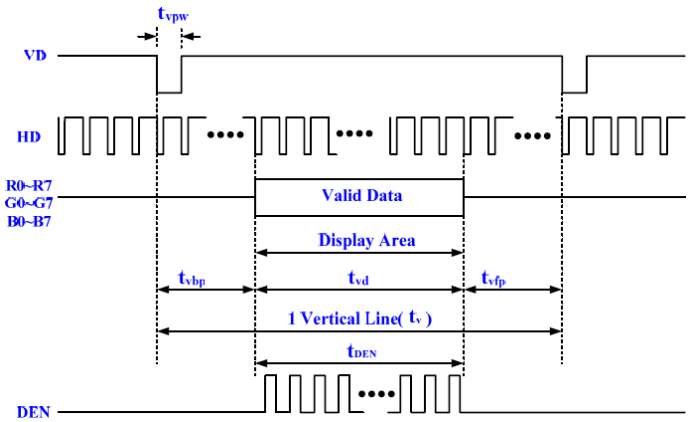


FIGURE 13. SPECIFICATIONS FOR THE VERTICAL TIMING

6 Software Design

Quartus II:

Open Quartus Prime and select ‘New Project Wizard’. An introduction prompt shows up and click on ‘Next’ after reading the instructions. Select the working directory for the current project and name it accordingly. The entity name is set as the name of the project by default. Then, click ‘Next’. Select the project type as per requirement and click on ‘Next’. You can make use of any of the existing files by adding them by file names when the following prompt appears. Else, click on ‘Next’. In the next prompt, select the package as FBGA, Pin count as 896 and Core speed grade as 6 from the respective drop-down menus. Once done, click on ‘Board’ tab. Select ‘DE1-SoC Board’ in the name column, check the box to create top-level design file and click ‘Next’. For the simulation tool type, select ‘ModelSim-Altera’ as the tool name and SystemVerilog as the format from the respective drop-down menus. Then, click ‘Next’. Go through the summary and click ‘Finish’ once done.

The ADC_SPI_Controller entity (Figure 33) handles the touch event on the LTM by triggering a pending interrupt for use by the NIOS processor. The LTM itself controls the timing of the ADC (physical IC built into the LTM) so the busy signal, pending IRQ, and ADC data must be sent to the controller for proper signal manipulation. The ADC_SPI_Controller also decodes the positional X and Y

coordinates sent to the FGPA from the LTM. These positions are used to determine which option the user has selected from the graphical user interface.

The LCD_SPI_controller (Figure 32) entity sends all the register settings to configure the LTM device. The settings are transmitted as 20 separate messages that are transmitted during the initialization of the device. Once all 20 messages have been sent, the SPI controller remains in a listening state until the next initialization event (reset).

The touch_irq_detector is the entity that will detect the inputs by the digitizer and increment the display mode value value. We use that change in value as the detected touch. We don't use the exact position of touvh for our projrct we just detect the touch at any point on the display.

The final entity used in the LTM instance was the LCD_Timing_Controller (Figure 35). The entity defines the display area, divides the area based on the image resolution and defines the RGB values for each pixel along with the display location and provides it as the output to the top-level entity which then displays image data on the LCD portion of the LTM. The RGB channels along with the horizontal and vertical synchronization and the data enable signals were controlled from this module. When data was 32-bits it was read from the SDRAM, the data was sent to this controller which output the data to the LTM module.

6.1 Application 1 – Chessboard Display

6.1.1 Design Approach:

For the first design implementation, the goal was to display a combination of different colors and create a chess board on the LCD display. When the screen detects a touch, then the image will switch to the next checkerboard pattern. There are a total of 7 different RGB checkerboard patterns.

6.1.2 Design Implementation:

For this design, there were many components required. The main components were the LCD SPI Controller, ADC SPI Controller, LCD Timing Controller, Touch IRQ Detector, and a counter along with the hex 7 segment component. The RTL Diagram is show below in Figure 11.

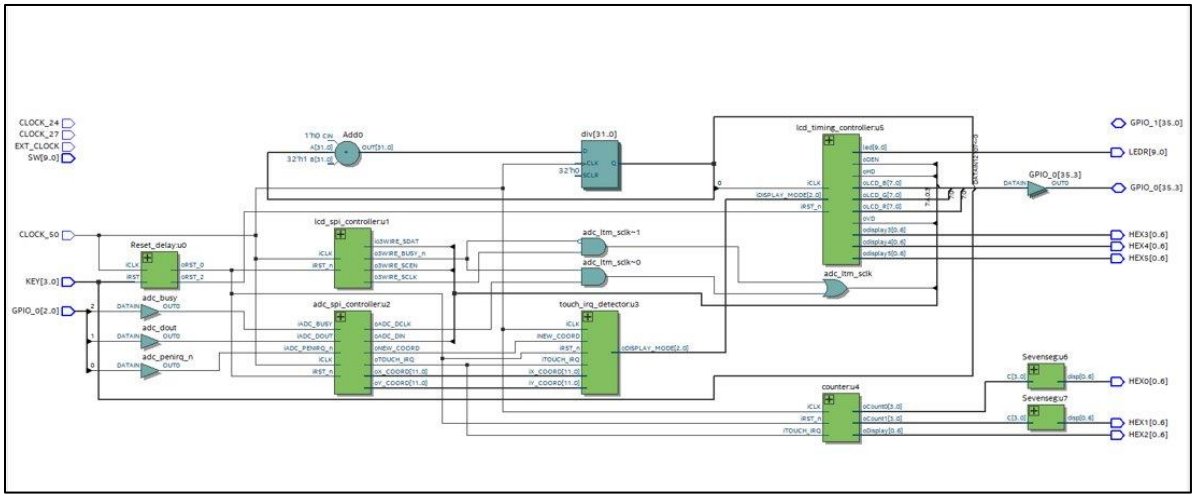


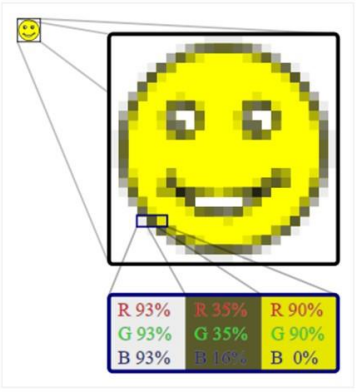
FIGURE 14. RTL VIEW OF DESIGN 1

The X and Y coordinates were calculated for the LCD display area. We chose to do a 9 x 9 grid, so the X and Y coordinates for the corresponding edges of each

block were found and calculated. These coordinates were then used to find out which color should be used for the specific coordinate.

The display mode will change each time the screen is tapped. There is a counter that will go through the different color modes that are preconfigured in the LCD Timing Controller. The different RGB hex values were written directly in the VHDL code as specific hex values. They were then written to the LCD screen and the values would differentiate depending on how many times the screen was tapped, eventually looping back to the original first picture.

23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
r7	r6	r5	r4	r3	r2	r1	r0	g7	g6	g5	g4	g3	g2	g1	g0	b7	b6	b5	b4	b3	b2	b1	b0



Offset (Dec)	Size (B)	Expected (Hex)	Description
0	2	"BM" (42 4D)	ID field
10	4	54 (36 00 00 00)	Pixel array offset
14	4	40 (28 00 00 00)	Header size
18	4	Read value	Image width in pixels
22	4	Read value	Image height in pixels
26	1	1 (01)	Number of color planes
28	1	24 (18)	Number of bits per pixel

FIGURE 15. RGB VALUES FROM THE IMAGE

6.2 Application 2 – BMP Image Display

6.2.1 Design Approach:

The goal for this second design similar to the first design, which was to display a an RGB image on the LCD. However this designed was more advanced, where instead of displaying simple RGB colors, two images were displayed one a 8 bit

color patter as shown in Figure 16 and a combination of 4 smiley images as shown in figure 17. Then once a touch was detected, images of other smiley .BMP's would circulate on the LCD screen until there are no touches displayed.

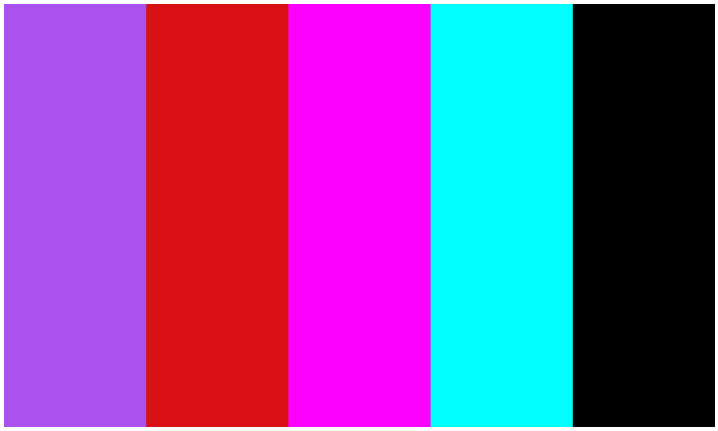


FIGURE 16. COLOR PALLET IMAGE



FIGURE 17. SMILEY FACE IMAGES

6.2.2 Design Implementation:

Similar components of the initial design were used for the implementation of this design.

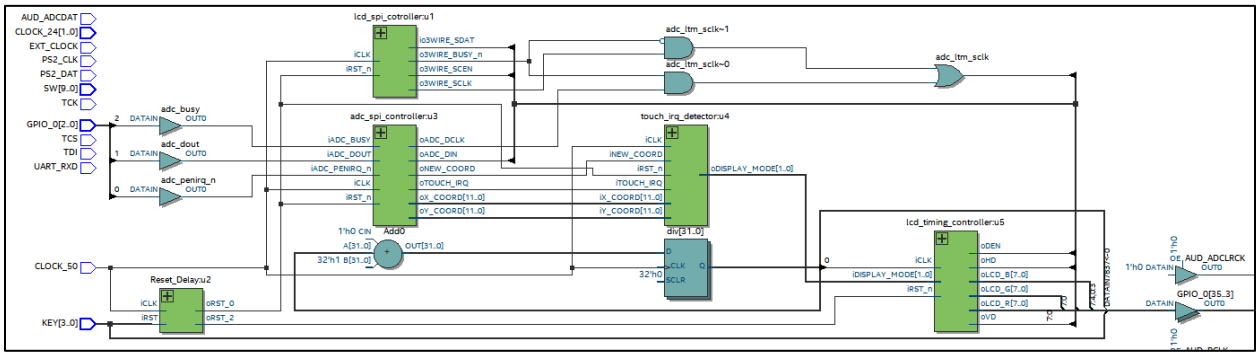


FIGURE 18. RTL VIEW OF DESIGN 2

But the LCD timing controller was updated to display the images in the LCD screen. Initially the images were converted into bmp format and the resolution of the image was set to 100x100. Using a matlab code, the RGB values of the images was extracted and converted into hexa decimal values so that it can be displayed in the LCD screen.

These RGB values were then inputted to the LCD timing controller code and stored locally in local variables. The display area of the LCD was divided into a 100x100 area. Using a counter the display area is looped from right to left and bottom to top and the respective RGB values of the images were sent to the LCD screen as outputs from the LTM timing controller board. The code is then executed and mounted on the board to display the images on the screen as shown in section 7.2.

6.3 Application 3 – Coordinate Tracker Display

6.3.1 Design Approach:

For this design, the goal was to display a red square on the LCD that would move relative to where the touch screen was touched. This again would make use of both the touch display hardware and the LCD display.

6.3.2 Design Implementation:

In order to achieve this, we must be able to detect the touch screen's coordinates, save the X-Y coordinates, and then use the coordinates in order to draw a red square on the LCD that would cover the general area of where the touch screen was touched.

This was done with similar main components as the first RGB chess design.

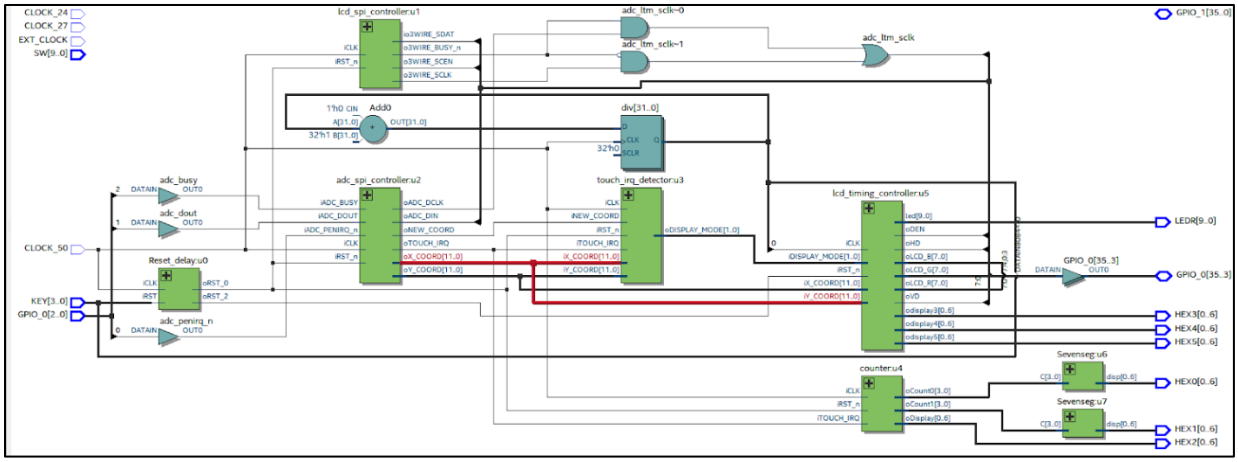


FIGURE 19. RTL VIEW OF DESIGN 3

After detecting and saving the X-Y coordinates of the touch screen, it is important to note that the LCD and Touch Screen's resolutions were different, and the

coordinates ran inversely. The touch screen's resolution for the X-coordinate was a 10-bit resolution which needed to be converted into the LCD's 9-bit Y-coordinate. Meanwhile the touch screen's Y-coordinate 10-bit resolution needed to be translated into the LCD's 10 bit X-coordinate. The inverse coordinates was done as an LCD scan correction since the LCD's display scan starts from right to left while the touch screen's sensors scanned from left to right.

After these coordinates translations were made, a red box was mapped around the LCD's translated coordinates. This allowed for the red box to follow the general area that the touch screen was being touched at. The number of touch detections were also displayed on the 7-seg display.

7 Results

The following images are captured to explain the operation of the proposed project. The first step of the process was for us to understand how we could display basic RGB colors on the LCD display panel, as shown in Fig. 17 and Fig.18. Once we were able to achieve this, the next step in the process was to convert and display the same RGB colors in a 10-bit array. Doing so allowed us to represent the color palette as a chess board as seen in Fig. 19 and 20. Post this, we were able to take premade BMP images and display them as a 4x1 array as shown in Fig. 21. Next, we took the same images premade images and displayed them as a 10-bit array on the LCD display panel. This is demonstrated in Figs. 22, 23, 24, 25.

7.1 RGB Color Display

The started code from the LTM documentation is updated in order to display the RGB pattern vertically as shown in figure 20 instead of the gradient configuration as shown in the Section 4 of the LTM manual. Once this was successful, the RGB chess board pattern was implemented by dividing the display area into a 9X9 resolution and displaying the RGB colors and the black and white in a chess board format as shown in figure 21 and 22.



FIGURE 20. VERTICAL DISPLAY OF THE RGB PATTERN

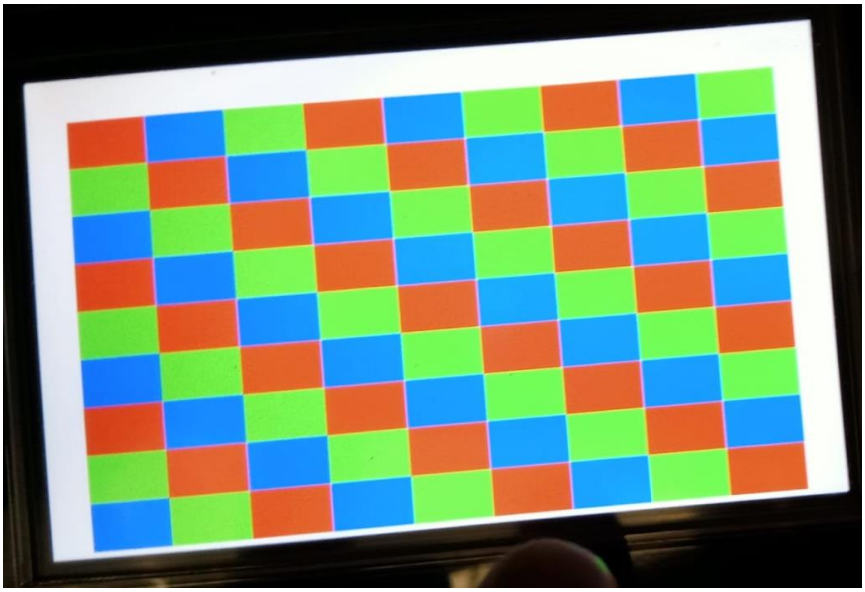


FIGURE 21. RGB COLOR PATTERN DISPLAY

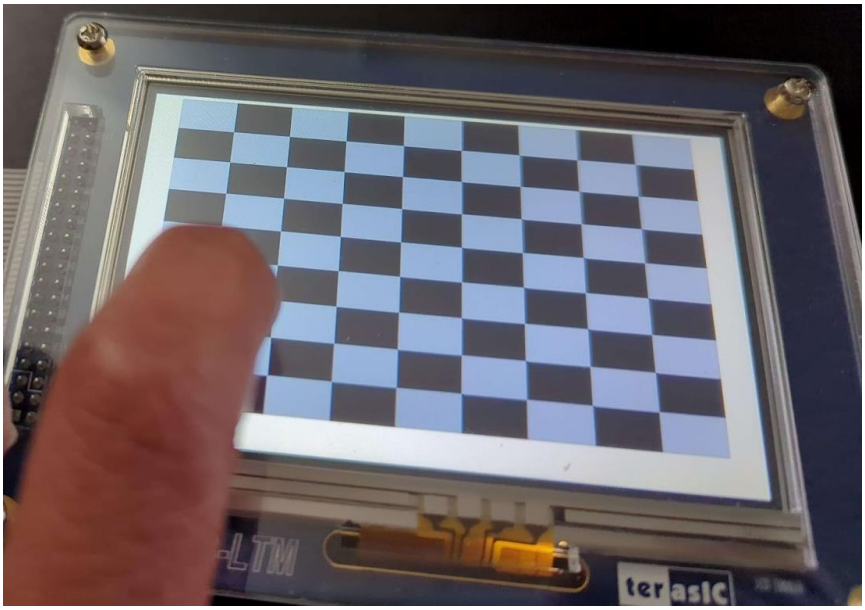


FIGURE 22. CHESS BOARD PATTERN DISPLAY

As this was completed, the next challenge was to display a 100x100 resolution image on the LCD screen. For this case we selected a color pallet image as shown in Figure 16 and then after dividing the display area into a 100x100 grid, the respective RGB values was sent into the LCD screen for display, as shown in Figure 23.

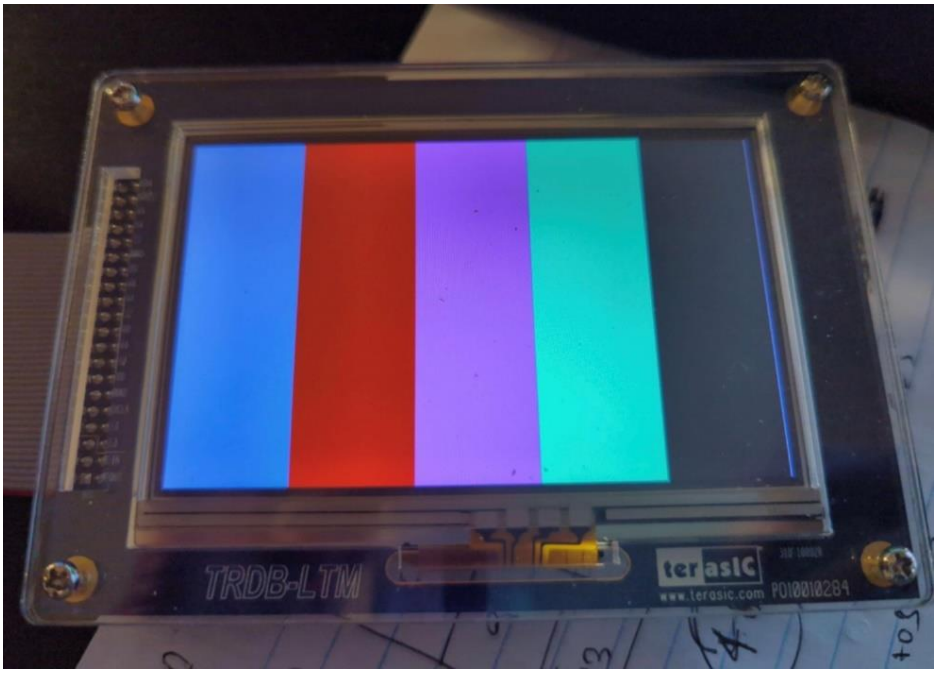


FIGURE 23. LCD DISPLAY OF COLOR PALLET

Demo Video Link:

7.2 BMP Image Displays

A similar approach was considered for the case of the smiley images. For this case we tried 2 cases, first we divided the screen into 4 parts vertically and displayed the smiley image 4 times in the screen as shown in figure 24.

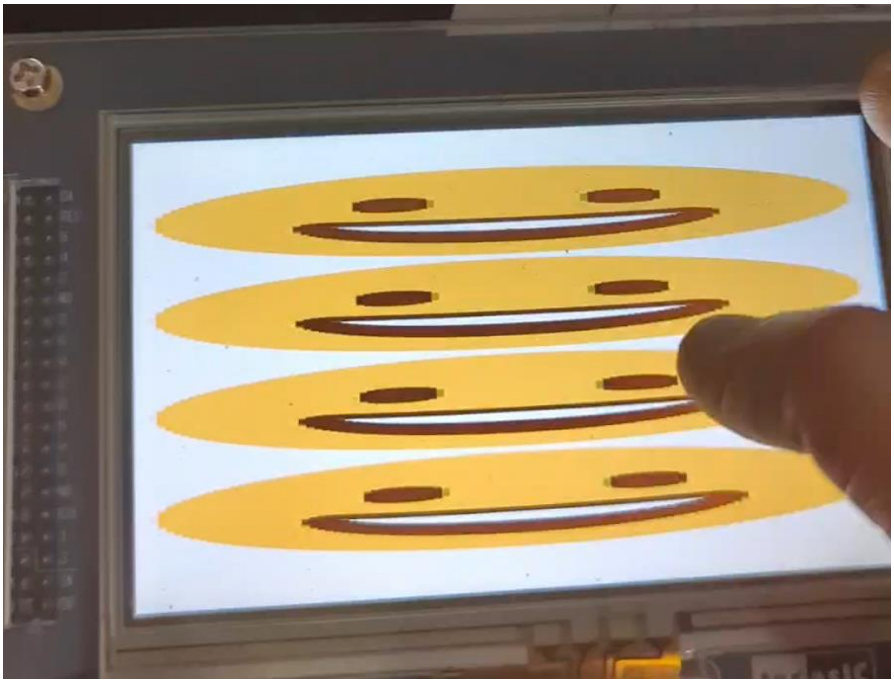


FIGURE 24. 1X4 DISPLAY OF THE SMILEY FACE

Then in case 2, we took the images as shown in figure 16 with a resolution of 100X400 extracted the RGB values and then using the counters sent the respective RGB values to the LCD grid and the images displayed are shown in Figure 25-28. In this case, as the screen is tapped the images, alternating between the 4 images as shown in Figure 17 If the screen is hold, the images keeps changing at a frequency of 50MHz iclock. When the Key 0 is placed the screen is set back to the initial image which is the green smiley face.

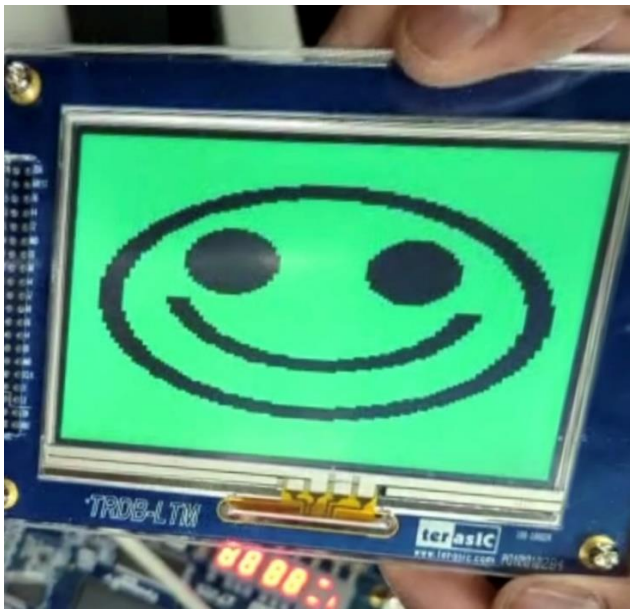


FIGURE 25. SMILEY FACE 1 DISPLAY IN LCD

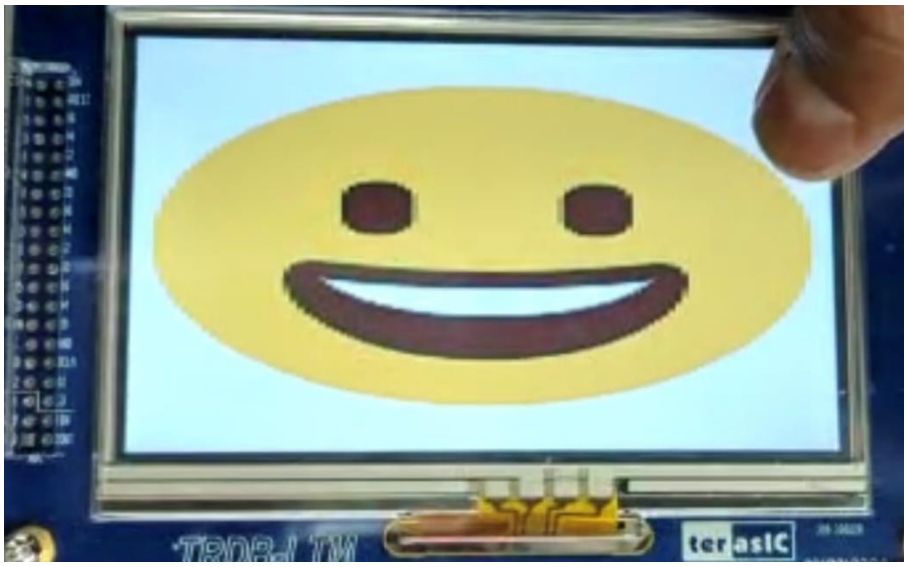


FIGURE 26. SMILEY FACE 2 DISPLAY IN LCD



FIGURE 27. SMILEY FACE 3 DISPLAY IN LCD



FIGURE 28. SMILEY FACE 4 DISPLAY IN LCD

The codes for all the patterns and the images are shown at the end of the document.

The code will also be shared in a zip format and uploaded to Moodle.

Demo Video Link:

https://www.youtube.com/shorts/eayxq-o6rh4

7.3 Coordinate Tracker Display

After flashing the board, the LCD screen starts off as all white. Then when there is a touch detected on the screen, a red box will appear and follow wherever the touchscreen is touched at. Figures 29, 30, and 31 show this functionality.

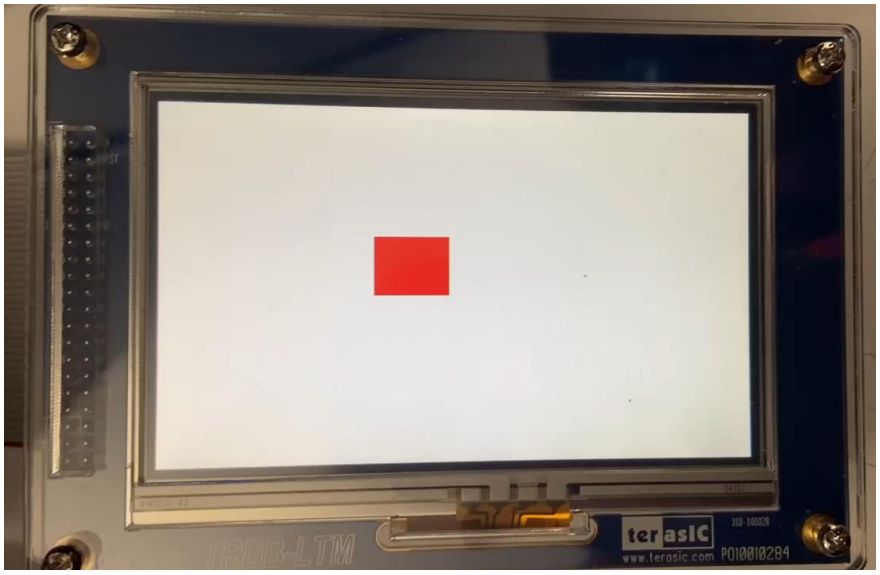


FIGURE 29. DESIGN 3 PICTURE 1

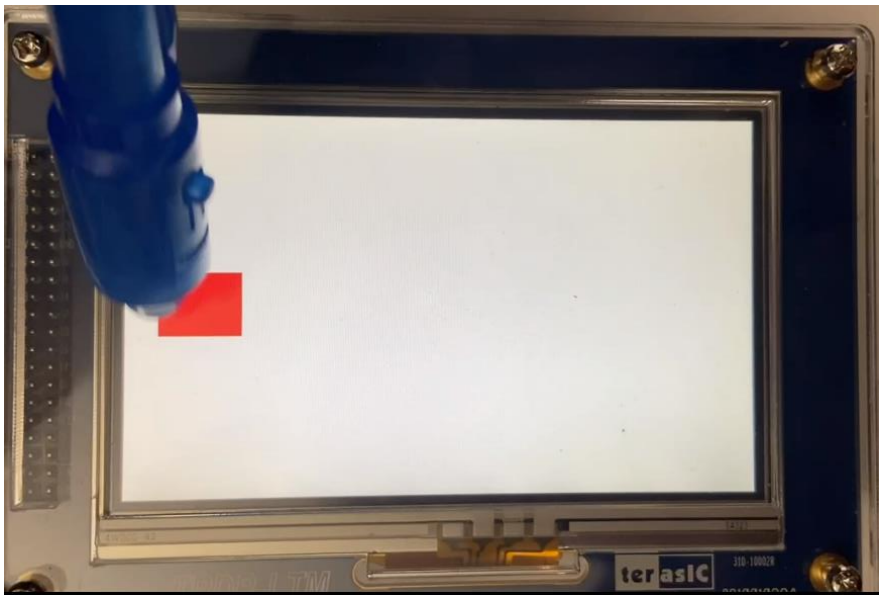


FIGURE 30. DESIGN 3 PICTURE 2

We can see that as the touch moves toward the edges, the red square moves slightly off center. This is due to the different bit resolutions between the LCD screen and the touchscreen detection as mentioned before.

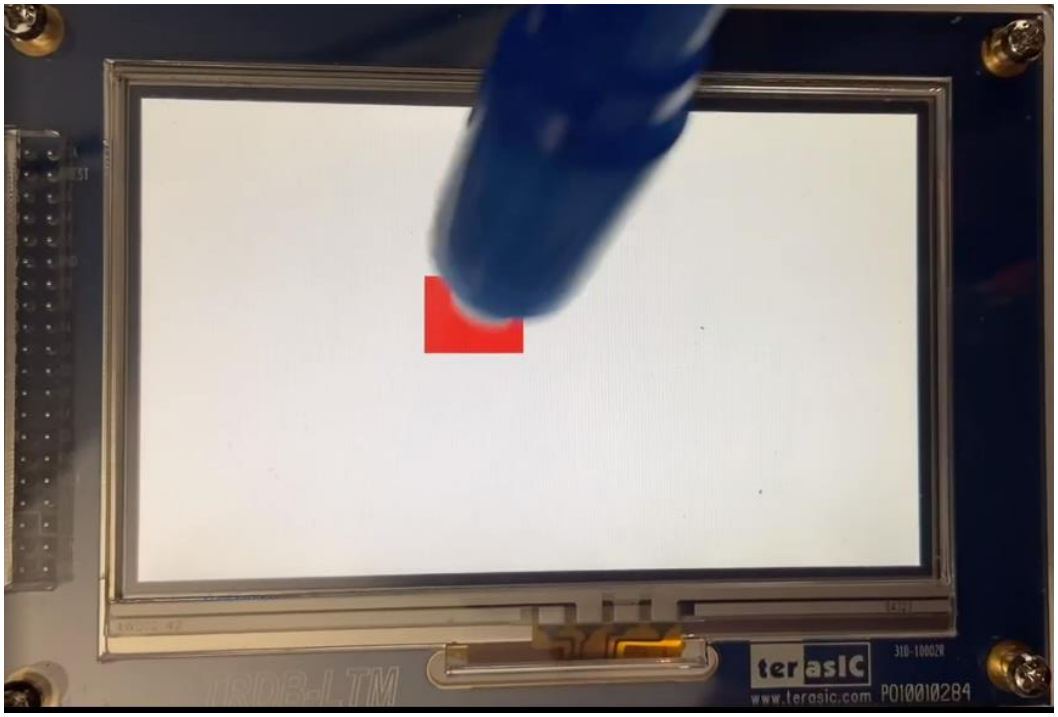


FIGURE 31. DESIGN 3 PICTURE 3

Demo Video Link:

https://www.youtube.com/shorts/yyLpgQqyZ0s

8 Conclusion

In this project we used the DE01-SoC board and made a Quartus Prime project with different entities to display to count number of touch inputs and to show the color displayed on the LCD screen in the seven-segment display.

The components that were required for this project are TRDB – LTM multi touch display from Terasic, DE01-SoC board, 20 pin connection wire and the software Quartus Prime 20.1 from intel. We use VHDL language to write the components of this project.

The project is useful for studying how the touch can be used in a LCD touch digitizer and how to show basic colors on the display via our software through FPGA board.

The future upgrades can be made to this project by displaying static images and using the touch for the control of showing next or previous images.

8.1 Challenges

While developing this project we faces several challenges. The first was that the NIOS II platform developer tool was unable to assist us in creating a soft processor. There was no useful supporting IP's available for the DE1-SOC parts. There was also no IP controller to help use the QSPI Flash component since it needs a 25 MHz clock. This led to errors at the time of code generation.

Another issue faced was when there was an attempt to use large arrays to hold 800x480 bmp files in RAM blocks. The compilation time took many hours and it never finished compiling in the end.

There was also the challenge of writing code without the basic operations such as division, multiplication, and shift operations of std_logic_vectors. The design had to be done with bit operations alone.

As mentioned before, the LCD and touch screen's coordinates were inverse and also scaled differently. This caused for the red box in design 3 to shift slightly and display below the detected touch area less accurately the further away from the center of the screen the touch was. This translation of coordinates was not conventional.

8.2 Future Work

In the future, it would be nice to be able to use the SDRAM component of the DE1-SOC board. This would require IP support with SDRAM controllers and PLL clock controllers. An alternative design project to do would be to be able to load a bmp image via the textio library as data and then load that data into RAM. Then an asynchronous read may output the RGB values from RAM memory onto the LTM screen. This architecture can be found in Figure 32.

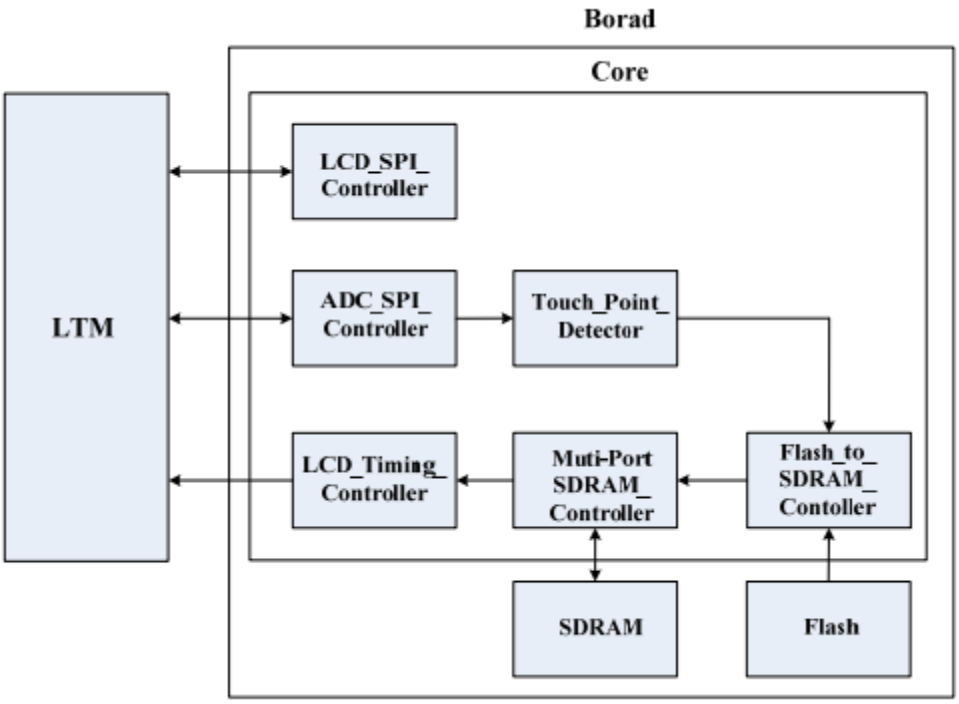


FIGURE 32. FUTURE ARCHITECTURE DESIGN INCLUDING SDRAM

9 References:

- https://www.intel.com/content/dam/alterawww/global/en_US/portal/dsn/42/doc-us-dsnbk-42-1004282204-de1-socuser-manual.pdf
- www.terasic.com
- https://ieeexplore.ieee.org/document/7033990
- www.altera.com

10 Code

GitHub: https://github.com/ppotturi-ou/EC5730_Final_Project

10.1 Project.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use IEEE.STD_LOGIC_unsigned.all;
use IEEE.STD_LOGIC_ARITH.ALL;
entity project  is
port (
-- Clock Input
CLOCK_24 : in std_logic;
CLOCK_27 : in std_logic; -- 27 MHz
CLOCK_50 : in std_logic; -- 50 MHz
EXT_CLOCK : in std_logic; --External Clock
--Push Button
KEY : in std_logic_vector(3 downto 0); --Pushbutton[3:0]
--DPDT Switch
SW : in std_logic_vector(9 downto 0); --Toggle Switch[9:0]
--7-SEG Dispaly
```

```
HEX0 : out std_logic_vector(0 to 6);
HEX1 : out std_logic_vector(0 to 6);
HEX2 : out std_logic_vector(0 to 6);
HEX3 : out std_logic_vector(0 to 6);
HEX4 : out std_logic_vector(0 to 6);
HEX5 : out std_logic_vector(0 to 6);
LEDR : out std_logic_vector(9 downto 0); -- LED Red[9:0]
--GPIO
GPIO_0 : inout std_logic_vector(35 downto 0); -- GPIO Connection 0
GPIO_1 : inout std_logic_vector(35 downto 0) -- GPIO Connection 1
);
end project;
architecture Top_Level of project is
----------Display Components-------------------------------------
component lcd_spi_controller is
port (
--Host side
iCLK: in STD_LOGIC; --Clock
iRST_n: in STD_LOGIC; -- Reset button
--3wire side
o3WIRE_SCLK : out STD_LOGIC;
io3WIRE_SDAT : inout STD_LOGIC;
o3WIRE_SCEN: out STD_LOGIC;
o3WIRE_BUSY_n : out STD_LOGIC
```

```
);
end component;
component adc_spi_controller is
port (
iCLK: in STD_LOGIC; -- Clock
iRST_n: in STD_LOGIC; -- Reset button
oADC_DIN : out STD_LOGIC;
oADC_DCLK : out STD_LOGIC;
oADC_CS : out STD_LOGIC;
iADC_DOUT: in STD_LOGIC;
iADC_BUSY: in STD_LOGIC;
iADC_PENIRQ_n : in STD_LOGIC;
oTOUCH_IRQ : out STD_LOGIC;
oX_COORD : out STD_LOGIC_VECTOR(11 downto 0);
oY_COORD : out STD_LOGIC_VECTOR(11 downto 0);
oNEW_COORD : out STD_LOGIC
);
end component;
component lcd_timing_controller is
port (
iCLK: in STD_LOGIC; -- LCD display clock
iRST_n: in STD_LOGIC; -- system reset
--LCD SIDE
oHD: out STD_LOGIC; -- LCD Horizontal sync
```

```
oVD: out STD_LOGIC; -- LCD Vertical sync
oDEN: out STD_LOGIC; -- LCD Data Enable
oLCD_R: out STD_LOGIC_VECTOR(7 downto 0); -- LCD Red color data
oLCD_G: out STD_LOGIC_VECTOR(7 downto 0); -- LCD Green color data
oLCD_B: out STD_LOGIC_VECTOR(7 downto 0); -- LCD Blue color data
iDISPLAY_MODE: in STD_LOGIC_VECTOR(1 downto 0);
led: out std_logic_vector(9 downto 0);
odisplay3 : out std_logic_vector(0 to 6);
odisplay4 : out std_logic_vector(0 to 6);
odisplay5 : out std_logic_vector(0 to 6)
);
end component;
component touch_irq_detector is
port (
iCLK: in STD_LOGIC;
iRST_n: in STD_LOGIC;
iTOUCH_IRQ : in STD_LOGIC;
iX_COORD : in STD_LOGIC_VECTOR(11 downto 0);
iY_COORD : in STD_LOGIC_VECTOR(11 downto 0);
iNEW_COORD : in STD_LOGIC;
oDISPLAY_MODE: out STD_LOGIC_vector(1 downto 0)
);
end component;
component Reset_delay is
```

```
port (
iCLK: in STD_LOGIC; -- System Clock
iRST: in STD_LOGIC; -- Reset (Active Low)
oRST_0: out STD_LOGIC; -- Reset is active for 0.042 sec longer
oRST_1: out STD_LOGIC; -- Reset is active for 0.063 sec longer
oRST_2: out STD_LOGIC -- Reset is active for 0.377 sec longer
);
end component;
component Sevenseg is
port
(
c                    : in std_logic_vector(3 downto 0);
disp     : out std_logic_vector(0 to 6)
);
end component;
component counter is
port (
iCLK: in STD_LOGIC;
iRST_n: in STD_LOGIC;
iTOUCH_IRQ : in STD_LOGIC;
oDisplay : out std_logic_vector(0 to 6);
oCount0 : out std_logic_vector(3 downto 0);
oCount1 : out std_logic_vector(3 downto 0)
);
```

```
end component;
----------END Display Components----------
signal DLY_RST_0 : std_logic;
signal DLY_RST_1 : std_logic;
signal DLY_RST_2 : std_logic;
=============================================================
-- Display Signal declarations
=============================================================
-- Touch panel signal
signal ltm_r: std_logic_vector(7 downto 0); -- LTM Red Data 8 Bits
signal ltm_g: std_logic_vector(7 downto 0); -- LTM Green Data 8 Bits
signal ltm_b: std_logic_vector(7 downto 0); -- LTM Blue Data 8 Bits
signal ltm_nclk : std_logic; -- LTM Clcok
signal ltm_hd : std_logic;
signal ltm_vd : std_logic;
signal ltm_den : std_logic;
signal ltm_grst : std_logic;
-- lcd 3wire interface
signal ltm_sclk : std_logic;
signal ltm_sda : std_logic;
signal ltm_scen : std_logic;
signal ltm_3wirebusy_n : std_logic;
-- Touch Screen Digitizer ADC
signal adc_dclk : std_logic;
```

```
signal adc_cs : std_logic;
signal adc_penirq_n : std_logic;
signal adc_busy : std_logic;
signal adc_din : std_logic;
signal adc_dout : std_logic;
signal adc_ltm_sclk : std_logic;
signal x_coord : std_logic_vector(11 downto 0);
signal y_coord : std_logic_vector(11 downto 0);
signal new_coord : std_logic;
signal display_mode : std_logic_vector(1 downto 0);
signal div : std_logic_vector(31 downto 0);
signal touch_irq : std_logic;
signal count0 :std_logic_vector(3 downto 0);
signal count1 :std_logic_vector(3 downto 0);
-- Zoom signals
signal mKJ: std_logic_vector(1 downto 0);
signal mZoom: std_logic;
-- SDRAM address info
signal mRD1_ADDR:std_logic_vector(22 downto 0);
signal mRD1_MAX_ADDR:std_logic_vector(22 downto 0);
signal mRD1_LENGTH:std_logic_vector(8 downto 0);
signal mRD1_LOAD:std_logic;
signal mRD2_ADDR:std_logic_vector(22 downto 0);
signal mRD2_MAX_ADDR:std_logic_vector(22 downto 0);
```

```
signal mRD2_LENGTH:std_logic_vector(8 downto 0);
signal mRD2_LOAD:std_logic;
signal mResetCCD,mCCD_reset: STD_LOGIC;
----------------------------------------
begin
u0 : Reset_Delay PORT MAP(
iCLK => CLOCK_50,
iRST => KEY(0),
oRST_0 => DLY_RST_0,
oRST_1 => DLY_RST_1,
oRST_2 => DLY_RST_2
);
----------DISPLAY----------
-- lcd 3 wire interface configuration
u1: lcd_spi_controller PORT MAP(
-- Host Side
iCLK => CLOCK_50,
iRST_n => DLY_RST_0,
-- 3wire Side
o3WIRE_SCLK => ltm_sclk,
io3WIRE_SDAT => ltm_sda,
o3WIRE_SCEN => ltm_scen,
o3WIRE_BUSY_n => ltm_3wirebusy_n
);
```

```
-- Touch Screen Digitizer ADC configuration
u2: adc_spi_controller PORT MAP(
iCLK => CLOCK_50,
iRST_n => DLY_RST_0,
oADC_DIN => adc_din,
oADC_DCLK => adc_dclk,
oADC_CS => adc_cs,
iADC_DOUT => adc_dout,
iADC_BUSY => adc_busy,
iADC_PENIRQ_n => adc_penirq_n,
oTOUCH_IRQ => touch_irq,
oX_COORD => x_coord,
oY_COORD => y_coord,
oNEW_COORD => new_coord
);
u3: touch_irq_detector PORT MAP(
iCLK => CLOCK_50,
iRST_n => DLY_RST_0,
iTOUCH_IRQ => touch_irq,
iX_COORD => x_coord,
iY_COORD => y_coord,
iNEW_COORD => new_coord,
oDISPLAY_MODE => display_mode
);
```

```
u4: counter PORT MAP(
iCLK => CLOCK_50,
iRST_n => DLY_RST_0,
iTOUCH_IRQ => touch_irq,
oDisplay => HEX2,
oCount0=>count0,
oCount1=>count1
);
-- LCD Display RGB data
u5: lcd_timing_controller PORT MAP(
iCLK => ltm_nclk,
iRST_n => DLY_RST_2,
-- lcd side
oHD => ltm_hd,
oVD => ltm_vd,
oDEN => ltm_den,
oLCD_R => ltm_r,
oLCD_G => ltm_g,
oLCD_B => ltm_b,
iDISPLAY_MODE => display_mode,
led=>LEDR,
odisplay3=>HEX3,
odisplay4=>HEX4,
odisplay5=>HEX5
```

```
);
u6:Sevenseg PORT MAP( c=>count0,
disp=>HEX0);
u7:Sevenseg PORT MAP( c=>count1,
disp=>HEX1);
=============================================================================
-- Structural coding
=============================================================================
-------Display--------------------
adc_penirq_n <= GPIO_0(0);
adc_dout <= GPIO_0(1);
adc_busy <= GPIO_0(2);
GPIO_0(3) <= adc_din;
GPIO_0(4) <= adc_ltm_sclk;
GPIO_0(5) <= ltm_b(3);
GPIO_0(6) <= ltm_b(2);
GPIO_0(7) <= ltm_b(1);
GPIO_0(8) <= ltm_b(0);
GPIO_0(9) <= ltm_nclk;
GPIO_0(10) <= ltm_den;
GPIO_0(11) <= ltm_hd;
GPIO_0(12) <= ltm_vd;
GPIO_0(13) <= ltm_b(4);
GPIO_0(14) <= ltm_b(5);
```

```
GPIO_0(15) <= ltm_b(6);
GPIO_0(16) <= ltm_b(7);
GPIO_0(17) <= ltm_g(0);
GPIO_0(18) <= ltm_g(1);
GPIO_0(19) <= ltm_g(2);
GPIO_0(20) <= ltm_g(3);
GPIO_0(21) <= ltm_g(4);
GPIO_0(22) <= ltm_g(5);
GPIO_0(23) <= ltm_g(6);
GPIO_0(24) <= ltm_g(7);
GPIO_0(25) <= ltm_r(0);
GPIO_0(26) <= ltm_r(1);
GPIO_0(27) <= ltm_r(2);
GPIO_0(28) <= ltm_r(3);
GPIO_0(29) <= ltm_r(4);
GPIO_0(30) <= ltm_r(5);
GPIO_0(31) <= ltm_r(6);
GPIO_0(32) <= ltm_r(7);
GPIO_0(33) <= ltm_grst;
GPIO_0(34) <= ltm_scen;
GPIO_0(35) <= ltm_sda;
adc_ltm_sclk <= '1' when (( adc_dclk = '1' AND ltm_3wirebusy_n = '1' ) OR (
ltm_3wirebusy_n = '0' AND ltm_sclk = '1' )) else '0';
ltm_nclk <= div(0); -- 25 Mhz
```

```
ltm_grst <= KEY(0);
process(CLOCK_50)
begin
if rising_edge(CLOCK_50) then
div <= div+1;
end if;
end process;
end Top_Level;
```

10.2 Touch_Irq_Detector

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;
entity touch_irq_detector is
port (
iCLK: in STD_LOGIC;
iRST_n: in STD_LOGIC;
iTOUCH_IRQ : in STD_LOGIC;
iX_COORD : in STD_LOGIC_VECTOR(11 downto 0);
iY_COORD : in STD_LOGIC_VECTOR(11 downto 0);
iNEW_COORD : in STD_LOGIC;
```

```
oDISPLAY_MODE: out STD_LOGIC_vector(1 downto 0)
);
end touch_irq_detector;

architecture IRQ_Detect of touch_irq_detector is
signal touch_en : std_logic;
signal touch_en_clr : std_logic;
signal touch_delay_cnt: std_logic_vector(24 downto 0);
signal mDISPLAY_MODE: STD_LOGIC_VECTOR(1 downto 0);
signal count : std_logic_vector(3 downto 0);
signal count1 : std_logic_vector(3 downto 0);
constant TOUCH_CNT_CLEAR : std_logic_vector(23 downto 0) := X"ffffff";
begin
process(iCLK, iRST_n)
begin
if (iRST_n = '0') then
touch_en <= '0';
elsif (iCLK'event and iCLK = '1') then
if (touch_en_clr = '1') then
touch_en <= '0';
end if;
if (iTOUCH_IRQ = '1') then
touch_en <= '1';
end if;
```

```
end if;
end process;
process(iCLK, iRST_n)
begin
if (iRST_n = '0') then
touch_delay_cnt <= (others => '0');
touch_en_clr <= '0';
elsif (iCLK'event and iCLK = '1') then
if (touch_delay_cnt = TOUCH_CNT_CLEAR) then
touch_delay_cnt <= (others => '0');
touch_en_clr <= '1';
elsif (touch_en = '1') then
touch_delay_cnt <= touch_delay_cnt + 1;
else
touch_delay_cnt <= (others => '0');
touch_en_clr <= '0';
end if;
end if;
end process;
process(iCLK, iRST_n)
variable carry : std_logic;
begin
if (iRST_n = '0') then
mDISPLAY_MODE <= "00";
```

```
elsif (iCLK'event and iCLK = '1') then
if (iTOUCH_IRQ = '1' AND touch_en = '0') then
mDISPLAY_MODE <= mDISPLAY_MODE + 1 ;

end if;
end if;
end process;
oDISPLAY_MODE <= mDISPLAY_MODE;
end IRQ_Detect;
```

10.3 Three Wire Controller

```
library ieee;
use ieee.std_logic_1164.all;
--use ieee.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;
use ieee.numeric_std.all;
entity three_wire_controller is
port (
iCLK: in STD_LOGIC; -- Clock
iRST: in STD_LOGIC; -- Reset button
iDATA: in STD_LOGIC_VECTOR(15 downto 0); -- DATA
iSTR: in STD_LOGIC; --
--HOST side
oACK: out STD_LOGIC;
```

```
oRDY: out STD_LOGIC;
oCLK: out STD_LOGIC;
--Serial Side
oSCEN: out STD_LOGIC;
SDA: inout STD_LOGIC; -- DATA
oSCLK: out STD_LOGIC
);
end three_wire_controller;
architecture Three_Cont of three_wire_controller is
-- Internal Register and Wire
signal mSPI_CLK: STD_LOGIC;
signal mSPI_CLK_DIV: STD_LOGIC_VECTOR(15 downto 0);
signal mSEN: STD_LOGIC;
signal mSDATA: STD_LOGIC;
signal mSCLK: STD_LOGIC;
signal mACK: STD_LOGIC;
signal mST: STD_LOGIC_VECTOR(4 downto 0);
-- Clock Setting
constant CLK_Freq : integer := 50000000; -- 50 MHz
constant SPI_Freq : integer := 20000; -- 20 KHz
begin
-- Serial Clock Generator
process(iCLK, iRST)
begin
```

```
if (iRST = '0') then
mSPI_CLK <= '0';
mSPI_CLK_DIV <= (others => '0');
elsif (iCLK'event and iCLK = '1') then
if( mSPI_CLK_DIV < (CLK_Freq/SPI_Freq) ) then
mSPI_CLK_DIV <= mSPI_CLK_DIV+1;
else
mSPI_CLK_DIV <= (others => '0');
mSPI_CLK <= not(mSPI_CLK);
end if;
end if;
end process;
-- Parallel to Serial
process(mSPI_CLK, iRST)
begin
if (iRST = '0') then
mSEN <= '1';
mSCLK <= '0';
mSDATA <= 'Z';
mACK <= '0';
mST <= "00000";
elsif(mSPI_CLK'event and mSPI_CLK = '0') then
if (iSTR = '1') then
if (mST < 17) then
```

```
mST <= mST + 1;
end if;
if (mST = 0) then
mSEN <= '0';
mSCLK <= '1';
elsif(mST = 8) then
mACK <= SDA;
elsif(mST = 16 AND mSCLK = '1') then
mSEN <= '1';
mSCLK <= '0';
end if;
if(mST < 16) then
mSDATA <= iDATA(to_integer(unsigned(15-mST)));
end if;
else
mSEN <= '1';
mSCLK <= '0';
mSDATA <= 'Z';
mACK <= '0';
mST <= "00000";
end if;
end if;
end process;
oACK <= mACK;
```

```
oRDY <= '1' when mST=17 else '0';
oSCEN <= mSEN;
oSCLK <= (mSCLK AND mSPI_CLK);
SDA <= 'Z' when mST=8 else 'Z' when mST=17 else mSDATA;
oCLK <= mSPI_CLK;
end Three_Cont;
```

10.4 Reset Delay

```
library ieee;
use ieee.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
entity Reset_delay is
port (
iCLK: in STD_LOGIC; -- System Clock
iRST: in STD_LOGIC; -- Reset (Active Low)
oRST_0: out STD_LOGIC; -- Reset is active for 0.042 sec longer
oRST_1: out STD_LOGIC; -- Reset is active for 0.063 sec longer
oRST_2: out STD_LOGIC -- Reset is active for 0.377 sec longer
);
end Reset_delay;
architecture RDEL of Reset_delay is
signal Cont : STD_LOGIC_VECTOR(31 downto 0);
begin
process(iCLK, iRST)
```

```
begin

if (iRST = '0') then -- If reset button is pushed in, clear the reset flags

Cont <= "00000000000000000000000000000000";

oRST_0 <= '0';

oRST_1 <= '0';

oRST_2 <= '0';

elsif rising_edge(iCLK) then

if(Cont /= X"11FFFFF") then -- Up counter runs up to Hex 11FFFFF(18874367) pulses. Clock input is 50 Mhz = 20 ns per clock pulse.

Cont <= Cont+1;

end if;

if(Cont >= X"1FFFFF") then -- RST_0 goes from 0 to 1 after 1FFFFF (2097151) pulses = 2097151*0.00000002 sec = 0.04194302 seconds

oRST_0 <= '1';

end if;

if(Cont >= X"2FFFFF") then -- RST_1 goes from 0 to 1 after 2FFFFF (3145727) pulses = 3145727*0.00000002 sec = 0.06291454 seconds

oRST_1 <= '1';

end if;

if(Cont >= X"11FFFFF") then -- RST_2 goes from 0 to 1 after 11FFFFF(18874367) pulses = 18874367*0.00000002 sec = 0.37748734 seconds

oRST_2 <= '1';

end if;

end if;

end process;

end RDEL;
```

10.5 Lcd Timing Controller (RGB and Chessboard Pattern)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;
use IEEE.numeric_std.all;
use std.textio.all;
use ieee.std_logic_textio.all;

entity lcd_timing_controller is

port
	(
		iCLK: in STD_LOGIC; -- LCD display clock
		iRST_n: in STD_LOGIC; -- system reset
		--LCD SIDE
		oHD: out STD_LOGIC; -- LCD Horizontal sync
		oVD: out STD_LOGIC; -- LCD Vertical sync
		oDEN: out STD_LOGIC; -- LCD Data Enable
		oLCD_R: out STD_LOGIC_VECTOR(7 downto 0); -- LCD Red color data
		oLCD_G: out STD_LOGIC_VECTOR(7 downto 0); -- LCD Green color data
		oLCD_B: out STD_LOGIC_VECTOR(7 downto 0); -- LCD Blue color data
		iDISPLAY_MODE: in STD_LOGIC_VECTOR(2 downto 0);
		led : out std_logic_vector(9 downto 0);
		odisplay3 : out std_logic_vector(0 to 6);
		odisplay4 : out std_logic_vector(0 to 6);
		odisplay5 : out std_logic_vector(0 to 6);
		oCount0 : out std_logic_vector(3 downto 0);
		oCount1 : out std_logic_vector(3 downto 0)
	);
end lcd_timing_controller;

architecture LCD_Tim of lcd_timing_controller is
```

```
--
//=======================================================================
=========
--// REG/SIgnal declarations
--
//=======================================================================
=========
        signal x_cnt: integer;
        signal y_cnt: integer;
        signal mred: std_logic_vector(7 downto 0);
        signal mgreen: std_logic_vector(7 downto 0);
        signal mblue: std_logic_vector(7 downto 0);
        signal display_area: std_logic;
        signal mhd: std_logic;
        signal mvd: std_logic;
        signal mden: std_logic;
        signal mREAD_SDRAM_EN: std_logic;
        signal msel: std_logic_vector(1 downto 0);
        signal mselx: std_logic_vector(1 downto 0); -- Update x axis
        signal msel1: std_logic_vector(1 downto 0);
        signal mselx1: std_logic_vector(1 downto 0); -- Update x axis
        signal msel2: std_logic_vector(1 downto 0);
        signal mselx2: std_logic_vector(1 downto 0); -- Update x axis
        --
//=======================================================================
=========
--// Color commination
--
//=======================================================================
=========
        signal red_1: std_logic_vector(7 downto 0);
        signal green_1: std_logic_vector(7 downto 0);
        signal blue_1: std_logic_vector(7 downto 0);
        signal red_2: std_logic_vector(7 downto 0);
        signal green_2: std_logic_vector(7 downto 0);
        signal blue_2: std_logic_vector(7 downto 0);
        signal red_3: std_logic_vector(7 downto 0);
        signal green_3: std_logic_vector(7 downto 0);
```

```
        signal blue_3: std_logic_vector(7 downto 0);
        signal red_4: std_logic_vector(7 downto 0);
        signal green_4: std_logic_vector(7 downto 0);
        signal blue_4: std_logic_vector(7 downto 0);
                --
//=================================================================
=========
--// Count touch
--
//=================================================================
=========
        signal count : std_logic_vector(3 downto 0);
        signal count1 : std_logic_vector(3 downto 0);
        signal graycnt: std_logic_vector(7 downto 0);
        signal graycnt1: std_logic_vector(7 downto 0); -- update gray




--
//=================================================================
========
--// Constant PARAMETER declarations
--
//=================================================================
========
        signal pattern_data: std_logic_vector(7 downto 0);
        constant H_LINE : integer := 1056;
        constant V_LINE : integer := 525;
        constant Hsync_Blank : integer := 216;
        constant Hsync_Front_Porch : integer := 40;
        constant Vertical_Back_Porch : integer := 35;
        constant Vertical_Front_Porch : integer := 10;
        constant num_of_combination : integer := 16;




--
```

```
--
//=======================================================================
=========
--// Structural coding
--
//=======================================================================
=========

Begin

        display_area <='1' when ((x_cnt>(Hsync_Blank-1) and -->215
                                                (x_cnt<(H_LINE-Hsync_Front_Porch))and --<
1016
                                                (y_cnt>(Vertical_Back_Porch-1)) and
                                                (y_cnt<(V_LINE - Vertical_Front_Porch))
                                                )) else '0';

--      ////////////////////////// x  y counter  and lcd hd generator //////////////////

process(iCLK, iRST_n)
        begin
                if (iRST_n = '0') then
                        x_cnt <= 0;
                        mhd <= '0';
                elsif (iCLK'event and iCLK = '1') then
                        if(x_cnt = (H_LINE-1)) then
                                x_cnt <= 0;
                                mhd <= '0';
                        else
                                x_cnt <= x_cnt + 1;

                                mhd <= '1';
                        end if;
                end if;
        end process;


process(iCLK, iRST_n)
begin
```

```
	if (iRST_n = '0') then
		y_cnt <= 0;
	elsif (iCLK'event and iCLK = '1') then
		if(x_cnt = (H_LINE-1)) then
			if(y_cnt = (V_LINE-1)) then
				y_cnt <= 0;
			else
				y_cnt <= y_cnt + 1;
			end if;
		end if;
	end if;
end process;




--	////////////////////////// touch panel timing ///////////////////

process(iCLK, iRST_n)
begin
	if (iRST_n = '0') then
			mvd  <= '1';
		elsif (y_cnt = 0) then
			mvd  <= '0';
		else
			mvd  <= '1';
		end if;
end process;

process(iCLK, iRST_n)
begin
	if (iRST_n = '0') then
			mden  <= '0';
		elsif (display_area='1') then
			mden  <= '1';
		else
			mden  <= '0';
		end if;
end process;
```

```
--//////////////RGB color patten generator ///////


msel <= "01" when (y_cnt>=40 and y_cnt <88) else "10" when (y_cnt>=88 and y_cnt<136) else
                                "11" when (y_cnt>=136         and y_cnt<184) else "01" when
(y_cnt>=184    and y_cnt<232)
                                else "10" when (y_cnt>=232  and y_cnt<280) else "11" when
(y_cnt>=280    and y_cnt<328)
                                else "01" when (y_cnt>=328  and y_cnt<376) else "10" when
(y_cnt>=376    and y_cnt<424)
                                else "11" when ((y_cnt>=424)          and (y_cnt<472)) else "00"
when (y_cnt>=472);


mselx <= "01" when (x_cnt>=256 and x_cnt<336) else "10" when (x_cnt>=336     and
x_cnt<416) else
                                "11" when (x_cnt>=416         and x_cnt<496) else "01" when
(x_cnt>=496    and x_cnt<576)
                                else "10" when (x_cnt>=576  and x_cnt<656) else "11" when
(x_cnt>=656    and x_cnt<736)
                                else "01" when (x_cnt>=736  and x_cnt<816) else "10" when
(x_cnt>=816    and x_cnt<896)
                                else "11" when (x_cnt>=896  and x_cnt<976) else "00" when
(x_cnt>=976);


-- Cordinate wise color
msel1 <= "01" when (y_cnt>=40 and y_cnt <88) else "10" when (y_cnt>=88 and y_cnt<136)
else
                                "01" when (y_cnt>=136         and y_cnt<184) else "10" when
(y_cnt>=184    and y_cnt<232)
                                else "01" when (y_cnt>=232  and y_cnt<280) else "10" when
(y_cnt>=280    and y_cnt<328)
                                else "01" when (y_cnt>=328  and y_cnt<376) else "10" when
(y_cnt>=376    and y_cnt<424)
                                else "01" when ((y_cnt>=424)          and (y_cnt<472)) else "00"
when (y_cnt>=472);
```

```
mselx1 <= "01" when (x_cnt>=256 and x_cnt<336) else "10" when (x_cnt>=336  and
x_cnt<416) else
                    "01" when (x_cnt>=416        and x_cnt<496) else "10" when
(x_cnt>=496   and x_cnt<576)
                    else "01" when (x_cnt>=576  and x_cnt<656) else "10" when
(x_cnt>=656   and x_cnt<736)
                    else "01" when (x_cnt>=736  and x_cnt<816) else "10" when
(x_cnt>=816   and x_cnt<896)
                    else "01" when (x_cnt>=896  and x_cnt<976) else "00" when
(x_cnt>=976);

msel2 <= "01" when (y_cnt>=40 and y_cnt <88) else "10" when (y_cnt>=88 and y_cnt<136)
else
                    "01" when (y_cnt>=136        and y_cnt<184) else "10" when
(y_cnt>=184   and y_cnt<232)
                    else "01" when (y_cnt>=232  and y_cnt<280) else "10" when
(y_cnt>=280   and y_cnt<328)
                    else "01" when (y_cnt>=328  and y_cnt<376) else "10" when
(y_cnt>=376   and y_cnt<424)
                    else "01" when ((y_cnt>=424)         and (y_cnt<472)) else "00"
when (y_cnt>=472);


mselx2 <= "01" when (x_cnt>=256 and x_cnt<336) else "10" when (x_cnt>=336  and
x_cnt<416) else
                    "01" when (x_cnt>=416        and x_cnt<496) else "10" when
(x_cnt>=496   and x_cnt<576)
                    else "01" when (x_cnt>=576  and x_cnt<656) else "10" when
(x_cnt>=656   and x_cnt<736)
                    else "01" when (x_cnt>=736  and x_cnt<816) else "10" when
(x_cnt>=816   and x_cnt<896)
                    else "01" when (x_cnt>=896  and x_cnt<976) else "00" when
(x_cnt>=976);


process(iCLK, iRST_n,mselx,msel)
        begin
```

```
if (iRST_n = '0') then
        pattern_data    <=      X"00";
elsif(mselx="00") then

                red_1 <= X"FF";
                green_1 <= X"FF";
                blue_1 <= X"FF";

                red_4 <= X"B4";
                green_4 <= X"8E";
                blue_4 <= X"45";
elsif(mselx="01") then

                if(msel="00") then


                        red_1 <= X"FF";
                        green_1 <= X"FF";
                        blue_1 <= X"FF";

                        red_4 <= X"B4";
                        green_4 <= X"8E";
                        blue_4 <= X"45";

                elsif (msel="10") then


                        red_1 <= X"00";
                        green_1 <= X"FF";
                        blue_1 <= X"00";

                        red_4 <= X"3E";
                        green_4 <= X"48";
                        blue_4 <= X"6F";

                elsif (msel="01") then


                        red_1 <= X"00";
```

```
                    green_1 <= X"00";
                    blue_1 <= X"FF";

                    red_4 <= X"6F";
                    green_4 <= X"3E";
                    blue_4 <= X"6F";

            elsif (msel="11") then


                    red_1 <= X"FF";
                    green_1 <= X"00";
                    blue_1 <= X"00";

                    red_4 <= X"FC";
                    green_4 <= X"0D";
                    blue_4 <= X"2A";
            end if;

    elsif(mselx="10") then

            if(msel="00") then


                    red_1 <= X"FF";
                    green_1 <= X"FF";
                    blue_1 <= X"FF";

                    red_4 <= X"B4";
                    green_4 <= X"8E";
                    blue_4 <= X"45";

            elsif (msel="01") then


                    red_1 <= X"00";
                    green_1 <= X"FF";
                    blue_1 <= X"00";
```

```
                    red_4 <= X"3E";
                    green_4 <= X"48";
                    blue_4 <= X"6F";

            elsif (msel="11") then


                    red_1 <= X"00";
                    green_1 <= X"00";
                    blue_1 <= X"FF";

                    red_4 <= X"6F";
                    green_4 <= X"3E";
                    blue_4 <= X"6F";
            elsif (msel="10") then


                    red_1 <= X"FF";
                    green_1 <= X"00";
                    blue_1 <= X"00";

                    red_4 <= X"FC";
                    green_4 <= X"0D";
                    blue_4 <= X"2A";
            end if;

        elsif(mselx="11") then

            if(msel="00") then


                    red_1 <= X"FF";
                    green_1 <= X"FF";
                    blue_1 <= X"FF";

                    red_4 <= X"B4";
                    green_4 <= X"8E";
                    blue_4 <= X"45";
            elsif (msel="11") then
```

```
                                red_1 <= X"00";
                                green_1 <= X"FF";
                                blue_1 <= X"00";

                                red_4 <= X"3E";
                                green_4 <= X"48";
                                blue_4 <= X"6F";

                        elsif (msel="10") then


                                red_1 <= X"00";
                                green_1 <= X"00";
                                blue_1 <= X"FF";

                                red_4 <= X"6F";
                                green_4 <= X"3E";
                                blue_4 <= X"6F";

                        elsif (msel="01") then


                                red_1 <= X"FF";
                                green_1 <= X"00";
                                blue_1 <= X"00";

                                red_4 <= X"FC";
                                green_4 <= X"0D";
                                blue_4 <= X"2A";
                        end if;

                else
                        pattern_data<=          X"00";
                end if;
end process;
--gray level patten generator
process(iCLK, iRST_n)
```

```
begin
	if (iRST_n = '0') then
		graycnt <= (others => '0');
	elsif((x_cnt>(Hsync_Blank-1))AND(x_cnt<(H_LINE-Hsync_Front_Porch))) then
			graycnt <= graycnt + 1;
	else
			graycnt <= (others => '0');
	end if;

end process;
process(iCLK, iRST_n)
	begin
		if (iRST_n = '0') then

			pattern_data    <=        X"00";

		elsif(mselx1="00") then

			graycnt1 <= X"ff";

		elsif(mselx1 ="10") then


				if(msel1 ="10") then

					graycnt1 <= X"d9";

				elsif (msel1 = "01") then

					graycnt1 <= X"00";

				else
					graycnt1 <= X"d9";
				end if;

		elsif(mselx1="01") then

				if(msel1 ="10") then
```

```
                                        graycnt1 <= X"00";


                                elsif (msel1 = "01") then

                                        graycnt1 <= X"d9";
                                else
                                        graycnt1 <= X"d9";

                                end if;
                        elsif(mselx1="11") then
                                if(msel1 ="10") then

                                        graycnt1 <= X"d9";


                                elsif (msel1 = "01") then

                                        graycnt1 <= X"00";


                                else
                                        graycnt1 <= X"d9";

                                end if;
                        else
                                pattern_data    <=      X"00";
                        end if;
                end process;


process(iCLK, iRST_n,mselx2,msel2)
                begin
                        if (iRST_n = '0') then
                                pattern_data    <=      X"00";
                        elsif(mselx2="00") then

                                red_2 <= X"FF";
                                green_2 <= X"FF";
```

```
                blue_2 <= X"FF";

                red_3 <= X"00";
                green_3 <= X"9B";
                blue_3 <= X"FF";

        elsif(mselx2="01") then

                if(msel2="00") then
                        red_3 <= X"00";
                        green_3 <= X"9B";
                        blue_3 <= X"FF";

                        red_2 <= X"FF";
                        green_2 <= X"FF";
                        blue_2 <= X"FF";

                elsif (msel2="10") then


                        red_2 <= X"00";
                        green_2 <= X"00";
                        blue_2 <= X"00";

                        red_3 <= X"E1";
                        green_3 <= X"22";
                        blue_3 <= X"28";

                elsif (msel2="01") then

                        red_2 <= X"FF";
                        green_2 <= X"3d";
                        blue_2 <= X"FF";

                        red_3 <= X"0A";
                        green_3 <= X"09";
                        blue_3 <= X"09";

                elsif (msel2="11") then
```

```
                red_2 <= X"F3";
                green_2 <= X"F3";
                blue_2 <= X"F3";

                red_3 <= X"09";
                green_3 <= X"0A";
                blue_3 <= X"0A";
        end if;

elsif(mselx2="10") then

        if(msel2="00") then


                red_2 <= X"FF";
                green_2 <= X"FF";
                blue_2 <= X"FF";

                red_3 <= X"00";
                green_3 <= X"9B";
                blue_3 <= X"FF";

        elsif (msel2="01") then


                red_2 <= X"00";
                green_2 <= X"00";
                blue_2 <= X"00";

                red_3 <= X"E1";
                green_3 <= X"22";
                blue_3 <= X"28";

        elsif (msel2="11") then


                red_2 <= X"00";
```

```
                    green_2 <= X"00";
                    blue_2 <= X"00";

                    red_3 <= X"E1";
                    green_3 <= X"22";
                    blue_3 <= X"28";

            elsif (msel2="10") then


                    red_2 <= X"FF";
                    green_2 <= X"3d";
                    blue_2 <= X"FF";

                    red_3 <= X"0A";
                    green_3 <= X"09";
                    blue_3 <= X"09";
            end if;

        elsif(mselx2="11") then

            if(msel2="00") then


                    red_2 <= X"FF";
                    green_2 <= X"FF";
                    blue_2 <= X"FF";

                    red_3 <= X"00";
                    green_3 <= X"9B";
                    blue_3 <= X"FF";

            elsif (msel2="11") then


                    red_2 <= X"00";
                    green_2 <= X"00";
                    blue_2 <= X"00";
```

```
                                        red_3 <= X"E1";
                                        green_3 <= X"22";
                                        blue_3 <= X"28";

                                elsif (msel2="10") then


                                        red_2 <= X"00";
                                        green_2 <= X"00";
                                        blue_2 <= X"00";

                                        red_3 <= X"E1";
                                        green_3 <= X"22";
                                        blue_3 <= X"28";

                                elsif (msel2="01") then


                                        red_2 <= X"F3";
                                        green_2 <= X"F3";
                                        blue_2 <= X"F3";
                                        red_3 <= X"09";
                                        green_3 <= X"0A";
                                        blue_3 <= X"0A";
                                end if;

                        else
                                pattern_data<=          X"00";
                        end if;
end process;

led(0)<=iDISPLAY_MODE(0);
                                led(1)<=iDISPLAY_MODE(1);
                                led(2)<=iDISPLAY_MODE(2);

--
--/////////////// displayed color pattern selection ///////////////
mred    <= X"28" when (iDISPLAY_MODE = "111") else red_3 when
                        (iDISPLAY_MODE = "011")else graycnt1 when
```

```
                    (iDISPLAY_MODE = "010") else red_1 when
                (iDISPLAY_MODE = "001") else red_4 when
                (iDISPLAY_MODE = "100") else red_4 when
                (iDISPLAY_MODE = "101") else red_2;

mgreen   <= X"8A" when (iDISPLAY_MODE = "111") else green_3 when
                (iDISPLAY_MODE = "011") else graycnt1 when
                    (iDISPLAY_MODE = "010") else green_1 when
                (iDISPLAY_MODE = "001") else green_4 when
                (iDISPLAY_MODE = "100") else green_3 when
                (iDISPLAY_MODE = "101") else green_2;

mblue    <= X"8D" when (iDISPLAY_MODE = "111")else blue_3 when
                (iDISPLAY_MODE = "011") else graycnt1 when
                    (iDISPLAY_MODE = "010") else blue_1 when
                (iDISPLAY_MODE = "001") else blue_4 when
                (iDISPLAY_MODE = "100") else blue_3 when
                (iDISPLAY_MODE = "101") else blue_2;

process(iCLK, iRST_n)

        begin
            if (iRST_n = '0') then

                        oHD    <= '0';
                        oVD    <= '0';
                        oDEN <= '0';
                        oLCD_R <= X"00";
                        oLCD_G <= X"00";
                        oLCD_B <= X"00";
                        odisplay5<="1111010";--r
                        odisplay4<="0110000";--E
                        odisplay3<="1000010";--d

            else

                        oHD    <= mhd;
                        oVD    <= mvd;
                        oDEN <= display_area;
```

```
                        oLCD_R <= mred;
                        oLCD_G <= mgreen;
                        oLCD_B <= mblue;
                        odisplay5<="1111010";--r
                        odisplay4<="0110000";--E
                        odisplay3<="1000010";--d
                end if;
        end process;
end LCD_Tim;
```

10.6 LCD Timing Controller (Displaying smiley image (100x400 resolution))

```
module lcd_timing_controller        (
                                    iCLK,                   // LCD display clock
                                    iRST_n,                 // systen reset
                                    //LCD SIDE
                                    oHD,                    // LCD Horizontal
sync
                                    oVD,                    // LCD Vertical sync
                                    oDEN,                   // LCD Data Enable
                                    oLCD_R,                         // LCD Red
color data
                                    oLCD_G,         // LCD Green color data
                                    oLCD_B,         // LCD Blue color data
                                    iDISPLAY_MODE
                                    );
//=============================================================================
========
// PARAMETER declarations
//=============================================================================
========
parameter H_LINE = 1056;
parameter V_LINE = 525;
parameter Hsync_Blank = 216;
parameter Hsync_Front_Porch = 40;
parameter Vertical_Back_Porch = 35;
parameter Vertical_Front_Porch = 10;
parameter hres_limit = 80;
```

```
parameter vres_limit = 48;
parameter tres_limit = 100;
//=============================================================================
=======
// PORT declarations
//=============================================================================
=======
input                   iCLK;
input                   iRST_n;
output  [7:0] oLCD_R;
output   [7:0]   oLCD_G;
output   [7:0]   oLCD_B;
output                  oHD;
output                  oVD;
output                  oDEN;
input   [1:0]    iDISPLAY_MODE;
//=============================================================================
=========
// REG/WIRE declarations
//=============================================================================
=========
integer pic_cnt = 0;
integer pic_cnt1 = 9;
integer res_cnt = 0;
integer vres_cnt = 0;
reg             [10:0]  x_cnt;
//reg           [10:0]  pic_cnt;
//reg           [10:0]  res_cnt;
reg             [9:0]    y_cnt;
wire    [7:0]    mred;
wire    [7:0]    mgreen;
wire    [7:0]    mblue;
wire                    display_area;
reg                             mhd;
reg                             mvd;
reg                             mden;
reg                             oHD;
reg                             oVD;
reg                             oDEN;
```

```
reg             [7:0]   oLCD_R;
reg             [7:0]   oLCD_G;
reg             [7:0]   oLCD_B;
wire    [1:0]   msel;
wire    [1:0]   mselx;
wire    [1:0]   msel1;
wire    [1:0]   mselx1;
reg             [7:0]   red_1;
reg     [7:0]   green_1;
reg     [7:0]   blue_1;
reg             [7:0]   red_2;
reg     [7:0]   green_2;
reg     [7:0]   blue_2;
reg     [7:0]   graycnt;
reg     [7:0]   graycnt1;
reg     [7:0]   pattern_data;
reg     [7:0]   pattern_data_B [0:99] = '{8'h00, 8'h00, 8'hFF, 8'hFF, 8'hFD, 8'hFD, 8'h00, 8'h00,
8'hFF, 8'hFF, 8'h00, 8'h00, 8'hFF, 8'hFF, 8'hFD, 8'hFD, 8'h00, 8'h00, 8'hFF, 8'hFF,8'h00, 8'h00,
8'hFF, 8'hFF, 8'hFD, 8'hFD, 8'h00, 8'h00, 8'hFF, 8'hFF,8'h00, 8'h00, 8'hFF, 8'hFF, 8'hFD, 8'hFD,
8'h00, 8'h00, 8'hFF, 8'hFF,8'h00, 8'h00, 8'hFF, 8'hFF, 8'hFD, 8'hFD, 8'h00, 8'h00, 8'hFF,
8'hFF,8'h00, 8'h00, 8'hFF, 8'hFF, 8'hFD, 8'hFD, 8'h00, 8'h00, 8'hFF, 8'hFF,8'h00, 8'h00, 8'hFF,
8'hFF, 8'hFD, 8'hFD, 8'h00, 8'h00, 8'hFF, 8'hFF,8'h00, 8'h00, 8'hFF, 8'hFF, 8'hFD, 8'hFD, 8'h00,
8'h00, 8'hFF, 8'hFF,8'h00, 8'h00, 8'hFF, 8'hFF, 8'hFD, 8'hFD, 8'h00, 8'h00, 8'hFF, 8'hFF,8'h00,
8'h00, 8'hFF, 8'hFF, 8'hFD, 8'hFD, 8'h00, 8'h00, 8'hFF, 8'hFF};
reg     [7:0]   pattern_data_G [0:99] = '{8'h00, 8'h00, 8'hFF, 8'hFF, 8'h00, 8'h00, 8'h13, 8'h13,
8'h5A, 8'h5A, 8'h00, 8'h00, 8'hFF, 8'hFF, 8'h00, 8'h00, 8'h13, 8'h13, 8'h5A, 8'h5A,8'h00, 8'h00,
8'hFF, 8'hFF, 8'h00, 8'h00, 8'h13, 8'h13, 8'h5A, 8'h5A,8'h00, 8'h00, 8'hFF, 8'hFF, 8'h00, 8'h00,
8'h13, 8'h13, 8'h5A, 8'h5A,8'h00, 8'h00, 8'hFF, 8'hFF, 8'h00, 8'h00, 8'h13, 8'h13, 8'h5A,
8'h5A,8'h00, 8'h00, 8'hFF, 8'hFF, 8'h00, 8'h00, 8'h13, 8'h13, 8'h5A, 8'h5A,8'h00, 8'h00, 8'hFF,
8'hFF, 8'h00, 8'h00, 8'h13, 8'h13, 8'h5A, 8'h5A,8'h00, 8'h00, 8'hFF, 8'hFF, 8'h00, 8'h00, 8'h13,
8'h13, 8'h5A, 8'h5A,8'h00, 8'h00, 8'hFF, 8'hFF, 8'h00, 8'h00, 8'h13, 8'h13, 8'h5A, 8'h5A,8'h00,
8'h00, 8'hFF, 8'hFF, 8'h00, 8'h00, 8'h13, 8'h13, 8'h5A, 8'h5A};
reg     [7:0]   pattern_data_R [0:99] = '{8'h00, 8'h00, 8'h01, 8'h01, 8'hFF, 8'hFF, 8'hD4, 8'hD4,
8'hA4, 8'hA4, 8'h00, 8'h00, 8'h01, 8'h01, 8'hFF, 8'hFF, 8'hD4, 8'hD4, 8'hA4, 8'hA4,8'h00, 8'h00,
8'h01, 8'h01, 8'hFF, 8'hFF, 8'hD4, 8'hD4, 8'hA4, 8'hA4,8'h00, 8'h00, 8'h01, 8'h01, 8'hFF, 8'hFF,
8'hD4, 8'hD4, 8'hA4, 8'hA4,8'h00, 8'h00, 8'h01, 8'h01, 8'hFF, 8'hFF, 8'hD4, 8'hD4, 8'hA4,
8'hA4,8'h00, 8'h00, 8'h01, 8'h01, 8'hFF, 8'hFF, 8'hD4, 8'hD4, 8'hA4, 8'hA4,8'h00, 8'h00, 8'h01,
8'h01, 8'hFF, 8'hFF, 8'hD4, 8'hD4, 8'hA4, 8'hA4,8'h00, 8'h00, 8'h01, 8'h01, 8'hFF, 8'hFF, 8'hD4,
```

```
8'hD4, 8'hA4, 8'hA4,8'h00, 8'h00, 8'h01, 8'h01, 8'hFF, 8'hFF, 8'hD4, 8'hD4, 8'hA4, 8'hA4,8'h00,
8'h00, 8'h01, 8'h01, 8'hFF, 8'hFF, 8'hD4, 8'hD4, 8'hA4, 8'hA4};

//=============================================================================
=========
// Structural coding
//=============================================================================
=========


// This signal indicate the lcd display area .
assign  display_area = ((x_cnt>(Hsync_Blank-1)&& //>215
                                                (x_cnt<(H_LINE-Hsync_Front_Porch))&& //<
1016
                                                (y_cnt>(Vertical_Back_Porch-1))&&
                                                (y_cnt<(V_LINE - Vertical_Front_Porch))
                                                ))  ? 1'b1 : 1'b0;


////////////////////////// x  y counter  and lcd hd generator //////////////////
always@(posedge iCLK or negedge iRST_n)
        begin
                if (!iRST_n)
                begin
                        x_cnt <= 11'd0;
                        mhd  <= 1'd0;
                end
                else if (x_cnt == (H_LINE-1))
                begin
                        x_cnt <= 11'd0;
                        mhd  <= 1'd0;
                end
                else
                begin
                        x_cnt <= x_cnt + 11'd1;
                        mhd  <= 1'd1;
                end
        end
```

```
always@(posedge iCLK or negedge iRST_n)
	begin
		if (!iRST_n)
			y_cnt <= 10'd0;
		else if (x_cnt == (H_LINE-1))
		begin
			if (y_cnt == (V_LINE-1))
			begin
				y_cnt <= 10'd0;
			end
			else
				y_cnt <= y_cnt + 10'd1;
		end
	end
always@(posedge iCLK or negedge iRST_n)
	begin
		if (!iRST_n)
		begin
			res_cnt = 0;
			vres_cnt = 0;
		end
		else if((x_cnt>(Hsync_Blank-1))&&(x_cnt<(H_LINE-Hsync_Front_Porch)))
		begin
			if((y_cnt>(Vertical_Back_Porch-1))&&(y_cnt<(V_LINE-
Vertical_Front_Porch)))
			begin
				if (res_cnt == (hres_limit-1))
					res_cnt = 0;
				else
					res_cnt = res_cnt + 1;
				if (x_cnt > (H_LINE-Hsync_Front_Porch-1))
				begin
					if (vres_cnt == (vres_limit-1))
						vres_cnt = 0;
					else
						vres_cnt = vres_cnt + 1;
				end
			end
		end
```

```
	end
always@(posedge iCLK or negedge iRST_n)
	begin
		if (!iRST_n)
		begin
			pic_cnt = 0;
			pic_cnt1 = 9;
		end
		else if((x_cnt>(Hsync_Blank-1))&&(x_cnt<(H_LINE-Hsync_Front_Porch)))
		begin
			if((y_cnt>(Vertical_Back_Porch-1))&&(y_cnt<(V_LINE-
Vertical_Front_Porch)))
			begin
				if (res_cnt == (hres_limit - 1))
				begin
					if (pic_cnt == (tres_limit - 1))
						pic_cnt = 0;
					else
						pic_cnt = pic_cnt + 1;
				end
				if (x_cnt > (H_LINE-Hsync_Front_Porch-1))
				begin
					if (vres_cnt ==(vres_limit-1))
					begin
						if (pic_cnt1 == tres_limit-1)
							pic_cnt1 = 9;
						else
							pic_cnt1 = pic_cnt1 + 10;
					end
					pic_cnt = pic_cnt1 - pic_cnt +1;
				end
			end
		end
	end
////////////////////////////// touch panel timing //////////////////

always@(posedge iCLK  or negedge iRST_n)
	begin
		if (!iRST_n)
```

```
                mvd  <= 1'b1;
            else if (y_cnt == 10'd0)
                mvd  <= 1'b0;
            else
                mvd  <= 1'b1;
        end

always@(posedge iCLK  or negedge iRST_n)
        begin
            if (!iRST_n)
                mden  <= 1'b0;
            else if (display_area)
                mden  <= 1'b1;
            else
                mden  <= 1'b0;
        end

//////////////RGB color patten generator ///////


assign  msel    =       (y_cnt>= 40 && y_cnt<88)   ?       2'b01:
                                        (y_cnt>=88    && y_cnt<136)       ?       2'b10
        :
                                        (y_cnt>=136   && y_cnt<184)       ?       2'b11
        :
                                        (y_cnt>=184   && y_cnt<232)       ?       2'b01
        :
                                        (y_cnt>=232   && y_cnt<280)       ?       2'b10
        :
                                        (y_cnt>=280   && y_cnt<328)       ?       2'b11
        :
                                        (y_cnt>=328   && y_cnt<376)       ?       2'b01
        :
                                        (y_cnt>=376   && y_cnt<424)       ?       2'b10
        :
                                        (y_cnt>=424   && y_cnt<472)       ?       2'b11
        :
                                        (y_cnt>=472   && y_cnt<520)       ?       2'b01
        :
```

```
            2'b00;
assign  mselx           =(x_cnt>=216 && x_cnt<296) ?      2'b01:
                                        (x_cnt>=296   && x_cnt<376)       ?       2'b10
        :
                                        (x_cnt>=376   && x_cnt<456)       ?       2'b11
        :
                                        (x_cnt>=456   && x_cnt<536)       ?       2'b01
        :
                                        (x_cnt>=536   && x_cnt<616)       ?       2'b10
        :
                                        (x_cnt>=616   && x_cnt<696)       ?       2'b11
        :
                                        (x_cnt>=696   && x_cnt<776)       ?       2'b01
        :
                                        (x_cnt>=776   && x_cnt<856)       ?       2'b10
        :
                                        (x_cnt>=856   && x_cnt<936)       ?       2'b11
        :
                                        (x_cnt>=936   && x_cnt<1016)      ?       2'b01
        :

            2'b00;
always@(posedge iCLK  or negedge iRST_n)
        begin
                if (!iRST_n)
                        pattern_data    <=      8'h00;
                else if(mselx==0)
                        begin
                                red_1 <= 8'hFF;
                                green_1 <= 8'hFF;
                                blue_1 <= 8'hFF;
                end
                else if(mselx==1)
                        begin
                                if(msel==0)
                                        begin
                                                red_1 <= 8'hFF;
                                                green_1 <= 8'hFF;
```

```
                        blue_1 <= 8'hFF;
                    end
                else if (msel==2)
                    begin
                        red_1 <= 0;
                        green_1 <= 8'hFF;
                        blue_1 <= 0;
                    end
                else if (msel==1)
                    begin
                        red_1 <= 0;
                        green_1 <= 0;
                        blue_1 <= 8'hFF;
                    end
                else if (msel==3)
                    begin
                        red_1 <= 8'hFF;
                        green_1 <= 0;
                        blue_1 <= 0;
                    end
            end
        else if(mselx==2)
            begin
                if(msel==0)
                    begin
                        red_1 <= 8'hFF;
                        green_1 <= 8'hFF;
                        blue_1 <= 8'hFF;
                    end
                else if (msel==1)
                    begin
                        red_1 <= 0;
                        green_1 <= 8'hFF;
                        blue_1 <= 0;
                    end
                else if (msel==3)
                    begin
                        red_1 <= 0;
                        green_1 <= 0;
```

```
                                        blue_1 <= 8'hFF;
                                end
                        else if (msel==2)
                                begin
                                        red_1 <= 8'hFF;
                                        green_1 <= 0;
                                        blue_1 <= 0;
                                end
                end
                else if(mselx==3)
                begin
                        if(msel==0)
                                begin
                                        red_1 <= 8'hFF;
                                        green_1 <= 8'hFF;
                                        blue_1 <= 8'hFF;
                                end
                        else if (msel==3)
                                begin
                                        red_1 <= 0;
                                        green_1 <= 8'hFF;
                                        blue_1 <= 0;
                                end
                        else if (msel==2)
                                begin
                                        red_1 <= 0;
                                        green_1 <= 0;
                                        blue_1 <= 8'hFF;
                                end
                        else if (msel==1)
                                begin
                                        red_1 <= 8'hFF;
                                        green_1 <= 0;
                                        blue_1 <= 0;
                                end
                end
                else
                        pattern_data<=          8'h00;
        end
```

```
//////////////Picture generator ///////

always@(posedge iCLK  or negedge iRST_n)
        begin
                if (!iRST_n)
                        pattern_data    <=      8'h00;
                else if((x_cnt>(Hsync_Blank-1))&&(x_cnt<(H_LINE-Hsync_Front_Porch)))
                begin
                        if((y_cnt>(Vertical_Back_Porch-1))&&(y_cnt<(V_LINE-
Vertical_Front_Porch)))
                        begin
                                red_2 <= pattern_data_R[pic_cnt];
                                green_2 <= pattern_data_G[pic_cnt];
                                blue_2 <= pattern_data_B[pic_cnt];
                        end
                end
end

//////////////gray level  patten generator ///////

always@(posedge iCLK  or negedge iRST_n)
        begin
                if (!iRST_n)
                        graycnt <= 0;
                else if((x_cnt>(Hsync_Blank-1))&&(x_cnt<(H_LINE-Hsync_Front_Porch)))
                        graycnt <= graycnt + 1;
                else
                        graycnt <= 0;
end

//////////////chessboard pattern ///////

assign  msel1  =        (y_cnt>= 40 && y_cnt<88)   ?        2'b01:
                                                (y_cnt>=88    && y_cnt<136)        ?        2'b10
        :
                                                (y_cnt>=136   && y_cnt<184)        ?        2'b01
        :
```

```
                                        (y_cnt>=184   && y_cnt<232)       ?       2'b10
        :
                                        (y_cnt>=232   && y_cnt<280)       ?       2'b01
        :
                                        (y_cnt>=280   && y_cnt<328)       ?       2'b10
        :
                                        (y_cnt>=328   && y_cnt<376)       ?       2'b01
        :
                                        (y_cnt>=376   && y_cnt<424)       ?       2'b10
        :
                                        (y_cnt>=424   && y_cnt<472)       ?       2'b01
        :
                                        (y_cnt>=472   && y_cnt<520)       ?       2'b10
        :

          2'b00;
assign  mselx1=  (x_cnt>=216 && x_cnt<296)  ?    2'b01:
                                        (x_cnt>=296   && x_cnt<376)       ?       2'b10
        :
                                        (x_cnt>=376   && x_cnt<456)       ?       2'b01
        :
                                        (x_cnt>=456   && x_cnt<536)       ?       2'b10
        :
                                        (x_cnt>=536   && x_cnt<616)       ?       2'b01
        :
                                        (x_cnt>=616   && x_cnt<696)       ?       2'b10
        :
                                        (x_cnt>=696   && x_cnt<776)       ?       2'b01
        :
                                        (x_cnt>=776   && x_cnt<856)       ?       2'b10
        :
                                        (x_cnt>=856   && x_cnt<936)       ?       2'b01
        :
                                        (x_cnt>=936   && x_cnt<1016)      ?       2'b10
        :

          2'b00;

always@(posedge iCLK  or negedge iRST_n)
```

```
	begin
		if (!iRST_n)
			graycnt1 <= 0;
		else if(mselx1==0)
			graycnt1 <= 8'h7ff;
		else if(mselx1 ==2)
			begin
				if(msel1 ==2)
					begin
						graycnt1 <= 8'h7f;
				end
				else if (msel1 == 1)
					begin
						graycnt1 <= 0;
				end
				else
					graycnt1 <= 8'hff;
		end
		else if(mselx1 == 1)
				begin
				if(msel1 == 2)
					begin
						graycnt1 <= 0;

				end
				else if (msel1 == 1)
					begin
						graycnt1 <= 8'h7f;

				end
				else
					graycnt1 <= 8'hff;
		end
		else
			graycnt1 <= 0;
end
////////////// displayed color pattern selection //////////////

assign mred    = (iDISPLAY_MODE == 2'b11)? red_2:
```

```
                        (iDISPLAY_MODE == 2'b10)?  graycnt1:
                    (iDISPLAY_MODE == 2'b01)?  red_1:
                                                        graycnt;

assign mgreen  = (iDISPLAY_MODE == 2'b11)?  green_2:
                    (iDISPLAY_MODE == 2'b10)?  graycnt1:
                        (iDISPLAY_MODE == 2'b01)?  green_1:
                                                    graycnt;

assign mblue   = (iDISPLAY_MODE == 2'b11)?  blue_2:
                    (iDISPLAY_MODE == 2'b10)?  graycnt1:
                        (iDISPLAY_MODE == 2'b01)?  blue_1:
                                                    graycnt;


always@(posedge iCLK or negedge iRST_n)
    begin
        if (!iRST_n)
            begin
                oHD    <= 1'd0;
                oVD    <= 1'd0;
                oDEN <= 1'd0;
                oLCD_R <= 8'd0;
                oLCD_G <= 8'd0;
                oLCD_B <= 8'd0;
            end
        else
            begin
                oHD    <= mhd;
                oVD    <= mvd;
                oDEN <= display_area;
                oLCD_R <= mred;
                oLCD_G <= mgreen;
                oLCD_B <= mblue;
            end
    end

endmodule
```

10.7 Lcd SPI Controller

```
library ieee;
use ieee.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
entity lcd_spi_controller is
port (
--Host side
iCLK: in STD_LOGIC; --Clock
iRST_n: in STD_LOGIC; -- Reset button
--3wire side
o3WIRE_SCLK : out STD_LOGIC;
io3WIRE_SDAT : inout STD_LOGIC;
o3WIRE_SCEN: out STD_LOGIC;
o3WIRE_BUSY_n : out STD_LOGIC
);
end lcd_spi_controller;
architecture SPI_Contrl of lcd_spi_controller is
component three_wire_controller is
port (
iCLK: in STD_LOGIC; -- Clock
iRST: in STD_LOGIC; -- Reset button
iDATA: in STD_LOGIC_VECTOR(15 downto 0); -- DATA
iSTR: in STD_LOGIC; --
--HOST side
```

```
oACK: out STD_LOGIC;
oRDY: out STD_LOGIC;
oCLK: out STD_LOGIC;
--Serial Side
oSCEN: out STD_LOGIC;
SDA: inout STD_LOGIC; -- DATA
oSCLK: out STD_LOGIC
);
end component;
--
================================================================
========
-- Signal declarations
--
================================================================
========
signal m3wire_str : STD_LOGIC;
signal m3wire_rdy : STD_LOGIC;
signal m3wire_ack : STD_LOGIC;
signal m3wire_clk : STD_LOGIC;
signal m3wire_data : STD_LOGIC_VECTOR(15 downto 0);
signal lut_data : STD_LOGIC_VECTOR(15 downto 0);
--signal lut_index : STD_LOGIC_VECTOR(5 downto 0);
signal lut_index : integer;
--signal msetup_st : STD_LOGIC_VECTOR(3 downto 0);
signal msetup_st : integer;
```

```
signal v_reverse : STD_LOGIC; --display Vertical reverse function
signal h_reverse : STD_LOGIC; --display Horizontal reverse function
constant g0 : STD_LOGIC_VECTOR(15 downto 0) := "0000000001101010"; --106
constant g1 : STD_LOGIC_VECTOR(15 downto 0) := "0000000011001000"; --200
constant g2 : STD_LOGIC_VECTOR(15 downto 0) := "0000000100100001"; --289
constant g3 : STD_LOGIC_VECTOR(15 downto 0) := "0000000101110111"; --375
constant g4 : STD_LOGIC_VECTOR(15 downto 0) := "0000000111001100"; --460
constant g5 : STD_LOGIC_VECTOR(15 downto 0) := "0000001000011111"; --543
constant g6 : STD_LOGIC_VECTOR(15 downto 0) := "0000001001110001"; --625
constant g7 : STD_LOGIC_VECTOR(15 downto 0) := "0000001011000001"; --705
constant g8 : STD_LOGIC_VECTOR(15 downto 0) := "0000001100010001"; --785
constant g9 : STD_LOGIC_VECTOR(15 downto 0) := "0000001101100000"; --864
constant g10 : STD_LOGIC_VECTOR(15 downto 0) := "0000001110101110"; --942
constant g11 : STD_LOGIC_VECTOR(15 downto 0) := "0000001111111100"; --1020
-- LUT Data Number
constant LUT_SIZE : integer := 20;
begin
three_wire_controller_inst : three_wire_controller PORT MAP (
iCLK => iCLK,
iRST => iRST_n,
iDATA => m3wire_data,
iSTR => m3wire_str,
oACK => m3wire_ack,
oRDY => m3wire_rdy,
```

```
oCLK => m3wire_clk,
oSCEN => o3WIRE_SCEN,
SDA => io3WIRE_SDAT,
oSCLK => o3WIRE_SCLK
);
process(m3wire_clk, iRST_n)
begin
if (iRST_n = '0') then
lut_index <= 0;
msetup_st <= 0;
m3wire_str <= '0';
o3WIRE_BUSY_n <= '0';
elsif (m3wire_clk'event and m3wire_clk = '1') then
if(lut_index<LUT_SIZE) then
o3WIRE_BUSY_n <= '0';
case msetup_st is
when 0 =>
msetup_st <= 1;
when 1 =>
msetup_st <= 2;
when 2 =>
m3wire_data <= lut_data;
m3wire_str <= '1';
msetup_st <= 3;
```

```
when 3 =>
if(m3wire_rdy = '1') then
if(m3wire_ack = '1') then
msetup_st <= 4;
else
msetup_st <= 0;
end if;
m3wire_str <= '0';
end if;
when 4 =>
lut_index <= lut_index+1;
msetup_st <= 0;
when others =>
null;
end case;
else
o3WIRE_BUSY_n <= '1';
end if;
end if;
end process;
--////////////////////// Config Data LUT //////////////////////////
process(LUT_INDEX)
begin
case LUT_INDEX is
```

```
when 0 =>
lut_data <= "010001" & "01" & g0(9 downto 8) & g1(9
downto 8) & g2(9 downto 8) & g3(9 downto 8);
when 1 =>
lut_data <= "010010" & "01" & g4(9 downto 8) & g5(9
downto 8) & g6(9 downto 8) & g7(9 downto 8);
when 2 =>
lut_data <= "010011" & "01" & g8(9 downto 8) & g9(9
downto 8) & g10(9 downto 8) & g11(9 downto 8);
when 3 =>
lut_data <= "010100" & "01" & g0(7 downto 0);
when 4 =>
lut_data <= "010101" & "01" & g1(7 downto 0);
when 5 =>
lut_data <= "010110" & "01" & g2(7 downto 0);
when 6 =>
lut_data <= "010111" & "01" & g3(7 downto 0);
when 7 =>
lut_data <= "011000" & "01" & g4(7 downto 0);
when 8 =>
lut_data <= "011001" & "01" & g5(7 downto 0);
when 9 =>
lut_data <= "011010" & "01" & g6(7 downto 0);
when 10 =>
```

```
lut_data <= "011011" & "01" & g7(7 downto 0);
when 11 =>
lut_data <= "011100" & "01" & g8(7 downto 0);
when 12 =>
lut_data <= "011101" & "01" & g9(7 downto 0);
when 13 =>
lut_data <= "011110" & "01" & g10(7 downto 0);
when 14 =>
lut_data <= "011111" & "01" & g11(7 downto 0);
when 15 =>
lut_data <= "010000" & "01" & "1111" & "0000";
when 16 =>
lut_data <= "010001" & "01" & "1111" & "0000";
when 17 =>
lut_data <= "000011" & "01" & "1101" & "1111";
when 18 =>
lut_data <= "000010" & "01" & "0000" & "0111";
when 19 =>
lut_data <= "000100" & "01" & "000101" & not(v_reverse)
& not(h_reverse);
when others =>
LUT_DATA <= X"0000";
end case;
end process;
```

```
h_reverse <= '1';
v_reverse <= '0';
end SPI_Contrl;
```

10.8 ADC SPI Controller

```
library ieee;
use ieee.std_logic_1164.all;
--use ieee.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;
entity adc_spi_controller is
port (
iCLK: in STD_LOGIC; --Clock
iRST_n: in STD_LOGIC; -- Reset button
oADC_DIN : out STD_LOGIC;
oADC_DCLK : out STD_LOGIC;
oADC_CS : out STD_LOGIC;
iADC_DOUT: in STD_LOGIC;
iADC_BUSY: in STD_LOGIC;
iADC_PENIRQ_n : in STD_LOGIC;
oTOUCH_IRQ : out STD_LOGIC; --Reference from main file
oX_COORD : out STD_LOGIC_VECTOR(11 downto 0);
oY_COORD : out STD_LOGIC_VECTOR(11 downto 0);
oNEW_COORD : out STD_LOGIC
);
```

```
end adc_spi_controller;


architecture ADC_SPI_Contrl of adc_spi_controller is
=============================================================================
-- Signal declarations
=============================================================================
signal d1_PENIRQ_n : std_logic;
signal d2_PENIRQ_n : std_logic;
signal touch_irq : std_logic;
signal dclk_cnt : std_logic_vector(15 downto 0);
signal dclk : std_logic;
signal transmit_en : std_logic;
signal spi_ctrl_cnt : std_logic_vector(6 downto 0);
signal mcs : std_logic;
signal mdclk : std_logic;
signal x_config_reg : std_logic_vector(7 downto 0);
signal y_config_reg : std_logic_vector(7 downto 0);
signal ctrl_reg : std_logic_vector(7 downto 0);
signal mdata_in : std_logic_vector(7 downto 0);
signal y_coordinate_config : std_logic;
signal eof_transmition : std_logic;
signal bit_cnt : std_logic_vector(5 downto 0);
signal madc_out : std_logic;
signal mx_coordinate: std_logic_vector(11 downto 0);
```

```
signal my_coordinate: std_logic_vector(11 downto 0);
signal rd_coord_strob: std_logic;
signal irq_cnt : std_logic_vector(5 downto 0);
signal clk_cnt : std_logic_vector(15 downto 0);
=============================================================
-- PARAMETER declarations
=============================================================
constant SYSCLK_FRQ : integer := 50000000;
constant ADC_DCLK_FRQ : integer := 1000;
constant ADC_DCLK_CNT : integer := SYSCLK_FRQ/(ADC_DCLK_FRQ*2);
begin
x_config_reg <= X"92";
y_config_reg <= X"d2";
process(iCLK, iRST_n)
begin
if (iRST_n = '0') then
madc_out <= '0';
elsif (iCLK'event and iCLK = '1') then
madc_out <= iADC_DOUT;
end if;
end process;
--pen irq detect
process(iCLK, iRST_n)
begin
```

```
if (iRST_n = '0') then
d1_PENIRQ_n <= '0';
d2_PENIRQ_n <= '0';
elsif (iCLK'event and iCLK = '1') then
d1_PENIRQ_n <= iADC_PENIRQ_n;
d2_PENIRQ_n <= d1_PENIRQ_n;
end if;
end process;
-- if iADC_PENIRQ_n form high to low , touch_irq goes high
touch_irq <= d2_PENIRQ_n AND not(d1_PENIRQ_n);
oTOUCH_IRQ <= touch_irq;
-- if touch_irq goes high , starting transmit procedure ,transmit_en goes high
-- if end of transmition and no penirq , transmit procedure stop.
process(iCLK, iRST_n)
begin
if (iRST_n = '0') then
transmit_en <= '0';
elsif (iCLK'event and iCLK = '1') then
if (eof_transmition = '1' AND iADC_PENIRQ_n = '1') then
transmit_en <= '0';
elsif (touch_irq = '1') then
transmit_en <= '1';
end if;
end if;
```

```
end process;
-- dclk_cnt
process(iCLK, iRST_n)
begin
if (iRST_n = '0') then
dclk_cnt <= (others => '0');
elsif (iCLK'event and iCLK = '1') then
if (transmit_en = '1') then
if (dclk_cnt = ADC_DCLK_CNT) then
dclk_cnt <= (others => '0');
else
dclk_cnt <= dclk_cnt + 1;
end if;
else
dclk_cnt <= (others => '0');
end if;
end if;
end process;
dclk <= '1' when dclk_cnt = ADC_DCLK_CNT else '0';
-- spi_ctrl_cnt
process(iCLK, iRST_n)
begin
if (iRST_n = '0') then
spi_ctrl_cnt <= (others => '0');
```

```
elsif (iCLK'event and iCLK = '1') then
if (dclk = '1') then
if (spi_ctrl_cnt = 65) then
spi_ctrl_cnt <= (others => '0');
else
spi_ctrl_cnt <= spi_ctrl_cnt + 1;
end if;
end if;
end if;
end process;
process(iCLK, iRST_n)
begin
if (iRST_n = '0') then
mcs <= '1';
mdclk <= '0';
mdata_in <= (others => '0');
y_coordinate_config <= '0';
mx_coordinate <= (others => '0');
my_coordinate <= (others => '0');
elsif (iCLK'event and iCLK = '1') then
if (transmit_en = '1') then
if (dclk = '1') then
if (spi_ctrl_cnt = 0) then
mcs <= '0';
```

```
mdata_in <= ctrl_reg;
elsif(spi_ctrl_cnt = 49) then
mdclk <= '0';
y_coordinate_config <= not(y_coordinate_config);
if (y_coordinate_config = '1') then
mcs <= '1';
else
mcs <= '0';
end if;
elsif(spi_ctrl_cnt /= 0) then
mdclk <= not(mdclk);
end if;
if (mdclk = '1') then
mdata_in <= mdata_in(6 downto 0) & '0';
elsif (mdclk = '0') then
if(rd_coord_strob = '1') then
if(y_coordinate_config = '1') then
my_coordinate <= my_coordinate(10 downto 0) & madc_out;
else
mx_coordinate <= mx_coordinate(10 downto 0) & madc_out;
end if;
end if;
end if;
end if;
```

```
end if;

end if;

end process;

oADC_CS <= mcs;

oADC_DIN <= mdata_in(7);

oADC_DCLK <= mdclk;

ctrl_reg <= y_config_reg when y_coordinate_config = '1' else x_config_reg;

eof_transmition <= '1' when (y_coordinate_config = '1' AND (spi_ctrl_cnt = 49) AND dclk ='1')
else '0';

rd_coord_strob <= '1' when ((spi_ctrl_cnt >=19)AND(spi_ctrl_cnt<=41)) else '0';

--X and Y coordinates

process(iCLK, iRST_n)

begin

if (iRST_n = '0') then

oX_COORD <= (others => '0');

oY_COORD <= (others => '0');

elsif (iCLK'event and iCLK = '1') then

if (eof_transmition = '1' AND my_coordinate /= 0) then

oX_COORD <= mx_coordinate;

oY_COORD <= my_coordinate;

end if;

end if;

end process;

--New coordinates

process(iCLK, iRST_n)
```

```
begin
if (iRST_n = '0') then
oNEW_COORD <= '0';
elsif (iCLK'event and iCLK = '1') then
if (eof_transmition = '1' AND my_coordinate /= 0) then
oNEW_COORD <= '1';
else
oNEW_COORD <= '0';
end if;
end if;
end process;
end ADC_SPI_Contrl;
```

10.9 Seven Segment

```
library ieee;
use ieee.std_logic_1164.all;
entity Sevenseg is
port
(
C : in std_logic_vector(3 downto 0);  --Logic Vector input value to check the value of count
disp : out std_logic_vector(0 to 6)          --SevenSegement display output to display the
count value
);
end Sevenseg;
```

```
architecture Behavioral of Sevenseg is
begin
process (c)
begin
case c is
        when "0000" => disp <= "0000001"; -- 0
        when "0001" => disp <= "1001111"; -- 1
        when "0010" => disp <= "0010010"; -- 2
        when "0011" => disp <= "0000110"; -- 3
        when "0100" => disp <= "1001100"; -- 4
        when "0101" => disp <= "0100100"; -- 5
        when "0110" => disp <= "0100000"; -- 6
        when "0111" => disp <= "0001111"; -- 7
        when "1000" => disp <= "0000000"; -- 8
        when "1001" => disp <= "0000100"; -- 9
        when "1010" => disp <= "0001000"; -- A
        when "1011" => disp <= "1100000"; -- B
        when "1100" => disp <= "0110001"; -- C
        when "1101" => disp <= "1000010"; -- D
        when "1110" => disp <= "0110000"; -- E
        when "1111" => disp <= "0111000"; -- F
        when others => disp <= "1111111";
        end case;
        end process;
```

```
end Behavioral;
```

10.10 Counter

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;
entity counter is
port (
iCLK: in STD_LOGIC;
iRST_n: in STD_LOGIC;
iTOUCH_IRQ : in STD_LOGIC;
oDisplay : out std_logic_vector(0 to 6);
oCount0 : out std_logic_vector(3 downto 0);  --To calculate the value of first hexadecimal count
oCount1 : out std_logic_vector(3 downto 0)  --To calculate the value of Second hexadecimal count
);
end counter;
architecture arch of counter is
signal count : std_logic_vector(3 downto 0);
signal count1 : std_logic_vector(3 downto 0);
signal touch_en : std_logic;
signal touch_en_clr : std_logic;
```

```
signal touch_delay_cnt: std_logic_vector(24 downto 0);
constant TOUCH_CNT_CLEAR : std_logic_vector(23 downto 0) := X"ffffff";
begin
process(iCLK, iRST_n)
begin
if (iRST_n = '0') then
touch_en <= '0';
elsif (iCLK'event and iCLK = '1') then
if (touch_en_clr = '1') then
touch_en <= '0';
end if;
if (iTOUCH_IRQ = '1') then
touch_en <= '1';
end if;
end if;
end process;
process(iCLK, iRST_n)
begin
if (iRST_n = '0') then
touch_delay_cnt <= (others => '0');
touch_en_clr <= '0';
elsif (iCLK'event and iCLK = '1') then
if (touch_delay_cnt = TOUCH_CNT_CLEAR) then
touch_delay_cnt <= (others => '0');
```

```
touch_en_clr <= '1';
elsif (touch_en = '1') then
touch_delay_cnt <= touch_delay_cnt + 1;
else
touch_delay_cnt <= (others => '0');
touch_en_clr <= '0';
end if;
end if;
end process;
--Counting the count values for each touch
process(iCLK, iRST_n)
begin
if (iRST_n = '0') then
count<="0000";
count1<="0000";
elsif (iCLK'event and iCLK = '1') then
if (iTOUCH_IRQ = '1' and touch_en = '0') then
count<= count+1;
if(count="1111") then
count<="0000";
count1<=count1+"0001";
end if;
end if;
end if;
```

```
end process;

oCount0<=count;

oCount1<=count1;

end arch;
```